HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ KHOA HỌC VÀ CÔNG NGHỆ

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: AN TOÀN VÀ BẢO MẬT HỆ THỐNG THÔNG TIN
MÃ HỌC PHẦN: INT1303

ĐỀ TÀI: GIẢI THUẬT BĂM SHA 0,1,2,3

Giảng viên hướng dẫn : Ths.Ninh Thị Thu Trang

Họ và tên sinh viên	Mã sinh viên
Phạm Quang Toản	B23DCCN835
Trần Văn Quang	B23DCCN695
Nguyễn Văn Bằng	B23DCCN067
Phạm Văn Kiên	B23DCCN466
Hoàng Hải	B23DCCN270

Hà Nội – 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	SV thực hiện	Thời hạn hoàn thành
1	Khái quát về hàm băm, giới thiệu SHA , SHA0	Phạm Quang Toản	18/3
2	SHA-1	Trần Văn Quang	18/3
3	SHA-2	Phạm Văn Kiên	18/3
4	SHA-3	Nguyễn Văn Bằng	18/3
5	Thử nghiệm mã hoá , thám mã	Hoàng Hải	18/3

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	SV thực hiện	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Phạm Quang Toản	5	5	5	5	5
2	Trần Văn Quang	5	5	5	5	5
3	Phạm Văn Kiên	5	5	5	5	5
4	Nguyễn Văn Bằng	5	5	5	5	5
5	Hoàng Hải	5	5	5	5	5

Ghi chú:

- Thái độ tham gia: Đánh giá điểm thái độ tham gia công việc chung của nhóm (từ 0: không tham gia, đến 5: chủ động, tích cực).
- Mức hoàn thành CV: Đánh giá điểm mức độ hoàn thành công việc được giao (từ 0: không hoàn thành, đến 5: hoàn thành xuất sắc).
- Kỹ năng giao tiếp: Đánh giá điểm khả năng tương tác, giao tiếp trong nhóm (từ 0: không hoặc giao tiếp rất yếu, đến 5: giao tiếp xuất sắc).
- Kỹ năng hợp tác: Đánh giá điểm khả năng hợp tác, hỗ trợ lẫn nhau, giải quyết mâu thuẫn, xung đột
- Kỹ năng lãnh đạo: Đánh giá điểm khả năng lãnh đạo (từ 0: không có khả năng lãnh đạo, đến 5: có khả năng lãnh đạo tốt, tổ chức và điều phối công việc trong nhóm hiệu quả).

MỤC LỤC

MỤC LỤC 3
DANH MỤC CÁC HÌNH VẼ 5
DANH MỤC CÁC BẢNG BIỂU 6
DANH MỤC CÁC TỪ VIẾT TẮT 6
MỞ ĐẦU 9
CHƯƠNG 1: KHÁI QUÁT VỀ HÀM BĂM VÀ GIẢI THUẬT SHA 9
1.1. Giới thiệu về hàm băm mật mã 10
1.2 Giới thiệu về họ thuật toán SHA 11
1.3 Kết chương 13
CHƯƠNG 2: GIẢI THUẬT SHA-0 14
2.1. Giới thiệu về SHA-0 14
2.2. Thuật toán SHA-0 15
2.5. Demo thực nghiệm phân tích lỗ hổng của SHA-0 16
2.6. Kết chương 17
CHƯƠNG 3: GIẢI THUẬT SHA-1 18
3.1 Giới thiệu SHA1 18
3.2. Cơ chế hoạt động 18
3.2.1. Tiền xử lý (Pre-processing) 18
3.2.2. Các vòng lặp xử lý: 19
3.2.3. Output: 20
3.3. SHA1 và Strict Avalanche Criterion (SAC) 20
3.3.1 Định nghĩa 20
3.3.2 Đồ thị 3D biểu diễn kết quả kiểm tra tính chất Strict Avalanche Criterion (SAC) của 21
3.4. Ưu và nhược điểm 21
3.4.1 Ưu điểm 21
3.4.2. Nhược điểm 22
3.5 Demo 22
3.5.1 Demo thực thi giải thuật mã hóa SHA1 22
3.5.2 Demo va chạm trên SHA-1: 23
3.6 Kết Chương 23
CHƯƠNG 4: GIẢI THUẬT SHA-2 24
4.1. Lịch sử và tiêu chuẩn 24
4.2. Tìm hiểu cấu trúc SHA-256: 25
4.2.1. Khái niệm: 25
4.2.2. Tính chất của hàm băm SHA-256: 25
4.2.3. Các bước thực hiện SHA-256: 26
4.3. Demo thực nghiệm tìm Collision (Tìm va chạm) của SHA-256: 29
4.3.1. Kết quả demo: 29
4.3.2. Kết luận rút ra: 30
4.4 Kết chương 30
CHƯƠNG 5: GIẢI THUẬT SHA-3 31
5.1. Giới thiệu SHA-3 31
5.1.1. Lịch sử ra đời 31
5.1.2. Các phiên bản 31

5.1.3. Ưu điểm và hạn chế..... 31
5.1.4 Ứng dụng..... 31
5.2. Kiến trúc giải thuật..... 32
5.2.1. Giới thiệu về State..... 32
5.2.2. Luồng hoạt động cấu trúc “Bọt biển” (Sponge Construction)..... 33
5.3. Giải thuật băm SHA3..... 33
5.3.1. Thu được mảng bit..... 33
5.3.2. Thêm padding và chia thành các block..... 34
5.3.3. Giai đoạn “Hấp thụ” (Absorbing Phase)..... 34
5.3.3.1. Bước XOR dữ liệu..... 34
5.3.3.2. Bước θ (theta)..... 34
5.3.3.3. Bước ρ (Rho)..... 36
5.3.3.4. Bước π (Pi)..... 38
5.3.3.5. Bước χ (Chi)..... 39
5.3.3.6. Bước ι (Iota)..... 40
5.3.4. Giai đoạn vắt (Squeezing Phase)..... 43
5.3.5. Xử lý và biểu diễn kết quả đầu ra..... 43
5.4. Demo..... 43
5.5 Kết chương..... 44
CHƯƠNG 6: CÁC ĐIỂM YẾU, CÁC DẠNG TẤN CÔNG VÀO SHA..... 44
6.1. Điểm yếu trong cơ chế mở rộng thông điệp (SHA-0)..... 45
6.2. Các dạng tấn công vào SHA-0..... 45
6.3. Tấn công Va chạm (Collision Attack) - "Tử huyệt" của SHA-1..... 46
6.4. Tấn công Mở rộng độ dài (Length Extension Attack) - Điểm yếu của SHA-2..... 46
6.5. Tấn công Vét cạn và Bảng cầu vồng (Brute Force & Rainbow Tables)..... 46
6.6. Tấn công Kênh bên (Side-Channel Attack)..... 46
6.7. Kết chương..... 47
CHƯƠNG 7: CÀI ĐẶT VÀ THỬ NGHIỆM MÃ HÓA VÀ THÁM MÃ CÁC GIẢI THUẬT SHA...48
7.1. Cài Đặt Môi Trường Thử Nghiệm..... 48
7.2. Thử Nghiệm Mã Hóa (Hashing)..... 48
7.2.1. Mã Nguồn và Thực Thi Hashing..... 48
7.2.2. Phân Tích Kết Quả Mã Hóa..... 49
7.3. Thử Nghiệm Thám Mã (Cracking)..... 49
7.3.1. Brute-Force Attack..... 49
7.3.2. Rainbow Table Attack..... 50
7.3.3. Collision Attack..... 50
7.4. Kết chương..... 51
KẾT LUẬN..... 53
1. Các kết quả đạt được..... 53
2. Hướng phát triển của đề tài..... 53
TÀI LIỆU THAM KHẢO..... 55

DANH MỤC CÁC HÌNH VẼ

Hình 1 - Minh hoạ kết quả phân tích lỗ hổng của SHA-0 16

Hình 2 - Lưu đồ xử lý 1 vòng của SHA1 18

Hình 3 - Đồ thị 3D biểu diễn kết quả kiểm tra tính chất Strict Avalanche Criterion 20

Hình 4 - Kết quả demo thực thi giải thuật SHA 22

Hình 5 - Kết quả demo tấn công va chạm mã SHA 23

Hình 6 - Lưu đồ một vòng lặp SHA-2 24

Hình 7 - Công thức biểu diễn nguyên lý hoạt động của SHA-2 25

Hình 8 - Bước nội bộ thứ j của hàm nén SHA-256 C 26

Hình 9 - Lịch trình thông điệp SHA-256 26

Hình 10 - Kết quả demo thử nghiệm tìm va chạm 29

Hình 11 - Mô tả các thành phần của state 32

Hình 12 - Sơ đồ hàm Bọt biển 33

Hình 13- Minh họa thu mảng bit của string “Hello world!” 34

Hình 14 - Trực quan hóa hàm C của θ 35

Hình 15 - Trực quan hóa hàm D của θ 35

Hình 16 - Trực quan hóa phép XOR đầu ra của θ 36

Hình 17 - Trực quan hóa hàm ρ trên một sheet của state. 37

Hình 18 - Minh họa phép nhân ma trận 38

Hình 19 - Suy diễn bảng offset của bước ρ. 38

Hình 20 - Trực quan hóa hàm π trên một slice. 39

Hình 21 - Sơ đồ hàm π trong tiêu chuẩn FIPS 202 39

Hình 22 - Trực quan hóa hàm χ trên một hàng. 40

Hình 23 - Trực quan hóa hàm ι trên một lane. 41

Hình 24 - Minh họa quá trình sinh Round Constant. 42

Hình 25 - Kết quả của thử nghiệm mã hóa 49

Hình 26 - kết quả của thử nghiệm thám mã brute-force 49

Hình 27 - kết quả của thử nghiệm thám mã rainbow table 50

Hình 28 - kết quả của thử nghiệm thám mã collision 51

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Giá trị r, c, d của SHA-3 32
Bảng 2. Bảng tổng hợp khả năng chống chịu tấn công 47
Bảng 3. Bảng so sánh chi tiết các thuật toán họ SHA 55

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh / Giải thích	Thuật ngữ tiếng Việt / Giải thích
SHA	Secure Hash Algorithm	Thuật toán băm an toàn
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia (Hoa Kỳ)
NSA	National Security Agency	Cơ quan An ninh Quốc gia (Hoa Kỳ)
FIPS	Federal Information Processing Standard	Tiêu chuẩn Xử lý Thông tin Liên bang
SHS	Secure Hash Standard	Tiêu chuẩn băm an toàn
IV	Initial Vector	Vectơ khởi tạo (Giá trị khởi đầu cho quá trình băm)
MAC	Message Authentication Code	Mã xác thực thông điệp
MDC	Modification Detection Code	Mã phát hiện sửa đổi
OWHF	One-way Hash Functions	Hàm băm một chiều
CRHF	Collision Resistant Hash Functions	Hàm băm kháng đụng độ (kháng va chạm)

SAC	Strict Avalanche Criterion	Tiêu chí khuếch tán nghiêm ngặt (Hiệu ứng tuyết lở)
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm
CPU	Central Processing Unit	Đơn vị xử lý trung tâm
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa (Dùng để tăng tốc thám mã)

MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, an toàn thông tin đã trở thành yếu tố then chốt quyết định sự tin cậy của các hệ thống lưu trữ và truyền tải dữ liệu. Giữa các kỹ thuật mật mã, hàm băm mật mã (Cryptographic Hash Functions) đóng vai trò là công cụ nền tảng không thể thay thế. Với khả năng ánh xạ dữ liệu có độ dài bất kỳ thành một "dấu vân tay" kỹ thuật số có độ dài cố định, hàm băm đảm bảo ba tính chất cốt lõi: tính kháng tiền ảnh, tính kháng tiền ảnh thứ hai và tính kháng đụng độ.

Họ thuật toán băm an toàn (Secure Hash Algorithm - SHA), được thiết kế bởi Cơ quan An ninh Quốc gia Hoa Kỳ (NSA) và tiêu chuẩn hóa bởi Viện Tiêu chuẩn và Công nghệ Quốc gia (NIST), đã trải qua một quá trình tiến hóa dài để đáp ứng các yêu cầu bảo mật ngày càng khắt khe. Từ phiên bản sơ khai SHA-0 với những khiếm khuyết trong cấu trúc tuyến tính, đến SHA-1 từng là tiêu chuẩn toàn cầu nhưng đã bị khuất phục bởi các cuộc tấn công va chạm thực nghiệm vào năm 2017. Sự chuyển dịch sang cấu trúc mạnh mẽ hơn của SHA-2 và sự ra đời của SHA-3 dựa trên kiến trúc "bọt biển" (Sponge construction) đánh dấu những bước ngoặt quan trọng trong nỗ lực chống lại sức mạnh tính toán ngày càng lớn của các hệ thống thám mã.

Báo cáo này tập trung phân tích chuyên sâu về lý thuyết, cấu trúc nội tại và cơ chế vận hành của họ thuật toán SHA từ phiên bản 0 đến 3. Nội dung nghiên cứu không chỉ dừng lại ở việc giải mã các phép toán logic, quá trình mở rộng thông điệp hay các vòng lặp nén phức tạp, mà còn đi sâu vào phân tích thực nghiệm các lỗ hổng bảo mật. Thông qua các kịch bản thám mã như tấn công vét cạn (Brute-force), tấn công ngày sinh nhật (Birthday attack) và tấn công va chạm (Collision attack), báo cáo làm rõ ranh giới giữa các thuật toán đã lỗi thời và các chuẩn mực bảo mật hiện đại.

Kết quả nghiên cứu và thực nghiệm trong báo cáo này cung cấp cái nhìn toàn diện về hiệu quả khuếch tán dữ liệu (hiệu ứng Avalanche), đồng thời khẳng định tầm quan trọng của việc lựa chọn giải pháp băm phù hợp trong việc xây dựng các hệ thống chữ ký số, xác thực mật khẩu và bảo vệ tính toàn vẹn dữ liệu trong môi trường mạng đầy biến động.

CHƯƠNG 1: KHÁI QUÁT VỀ HÀM BĂM VÀ GIẢI THUẬT SHA

1.1. Giới thiệu về hàm băm mật mã

- Hàm băm mật mã là một ánh xạ toán học

  $$: \{0,1\}^* \rightarrow \{0,1\}^n$$

  , biến đổi dữ liệu đầu vào có độ dài tùy ý thành chuỗi đầu ra có độ dài cố định, gọi là giá trị băm (hash value) hoặc message digest

- Về bản chất, giá trị băm đóng vai trò như một dấu vân tay số, cho phép biểu diễn rút gọn dữ liệu gốc trong khi vẫn duy trì khả năng nhận diện và kiểm chứng

Đặc trưng cốt lõi

- Tính đại diện rút gọn:
  Giá trị băm cho phép thay thế dữ liệu lớn bằng một biểu diễn ngắn gọn, giúp tối ưu hóa chi phí lưu trữ và tính toán
- Tính toàn vẹn dữ liệu:
  Việc so sánh hai giá trị băm cho phép phát hiện mọi thay đổi dù nhỏ nhất trong dữ liệu đầu vào
- Hỗ trợ xác thực và chữ ký số:
  Trong các hệ thống mật mã, giá trị băm được ký bằng khóa riêng, cho phép xác minh tính xác thực và nguồn gốc thông điệp

Mô hình toán học và yêu cầu cơ bản

- Hàm băm được mô hình hóa dưới dạng: h(x) , một hàm băm mật mã phải đảm bảo hai thuộc tính nền tảng:
  - Tính nén (Compression): ánh xạ không gian đầu vào lớn sang không gian đầu ra hữu hạn
  - Tính hiệu quả (Efficiency): tính toán nhanh, khả thi trong thực tế
- Hệ quả tất yếu của tính nén là tồn tại đụng độ (collision), do đó mục tiêu thiết kế là làm cho việc tìm collision trở nên bất khả thi về mặt tính toán

Cơ chế xử lý dữ liệu

- Thông điệp đầu vào được:
  - đệm (padding)
  - chia thành các khối kích thước cố định

    - xử lý tuần tự qua **hàm nén (compression function)**
- Quá trình này được lặp nhiều vòng, tạo thành một chuỗi biến đổi dẫn đến giá trị băm cuối cùng

Các tính chất bảo mật

- Kháng tiền ảnh (Preimage resistance): khó tìm x từ h(x)
- Kháng tiền ảnh thứ hai (Second preimage resistance): khó tìm x' khác x sao cho h(x')=h(x).
- Kháng đụng độ (Collision resistance):khó tìm x1 khác x2 sao cho h(x1)=h(x2)

Phân loại hàm băm

- Theo việc sử dụng khóa:
    - Không khóa: h(x) – dùng cho kiểm tra toàn vẹn
    - Có khóa: h(x,K) – dùng trong xác thực thông điệp
- Theo chức năng bảo mật:
    - MDC (Modification Detection Code): phát hiện thay đổi dữ liệu
    - MAC (Message Authentication Code): đảm bảo cả tính toàn vẹn và xác thực nguồn gốc

Mô hình xử lý chuẩn

- Phần lớn các hàm băm hiện đại dựa trên cấu trúc Merkle–Damgård
- Đặc điểm:
    - xử lý theo khối
    - lặp qua hàm nén
    - sử dụng chaining value giữa các vòng
- Mô hình này đảm bảo khả năng mở rộng và hiệu quả khi xử lý dữ liệu lớn

Ứng dụng thực tiễn

- Lưu trữ mật khẩu: tránh lưu plaintext, tăng an toàn khi rò rỉ dữ liệu
- Xác thực thông điệp: phát hiện chỉnh sửa hoặc giả mạo
- Kiểm tra toàn vẹn dữ liệu: trong truyền thông mạng và lưu trữ tệp
- Cấu trúc dữ liệu: sử dụng trong hash table để tối ưu truy xuất

1.2 Giới thiệu về họ thuật toán SHA

Secure Hash Algorithm (SHA) là một họ các thuật toán băm mật mã được tiêu chuẩn hóa trong bộ chuẩn Secure Hash Standard (SHS) của chính phủ Hoa Kỳ. Các thuật toán này được định nghĩa trong tiêu chuẩn FIPS 180 (Federal Information Processing Standard) và được sử dụng rộng rãi trong nhiều hệ thống an toàn thông tin như chữ ký số, xác thực dữ liệu, chứng chỉ số và các giao thức bảo mật trên Internet.

Mục tiêu của các thuật toán SHA là chuyển đổi một thông điệp đầu vào có độ dài tùy ý thành một chuỗi bit đầu ra có độ dài cố định, gọi là giá trị băm (hash value) hoặc thông điệp tóm tắt (message digest). Các thuật toán này được gọi là "an toàn" bởi vì chúng được thiết kế để thỏa mãn các tính chất bảo mật quan trọng của một hàm băm mật mã.

Theo tiêu chuẩn FIPS 180-2, được công bố ngày 1 tháng 8 năm 2002, một thuật toán băm được coi là an toàn nếu thỏa mãn các yêu cầu sau:

> "For a given algorithm, it is computationally infeasible:
> (1) to find a message that corresponds to a given message digest; or
> (2) to find two different messages that produce the same message digest.
> Any change to a message will, with a very high probability, result in a different message digest."

Có thể hiểu nội dung trên như sau:

1. Đối với một giá trị băm cho trước, việc tìm lại thông điệp ban đầu tương ứng là không khả thi về mặt tính toán.
2. Việc tìm ra hai thông điệp khác nhau nhưng có cùng giá trị băm là cực kỳ khó khăn về mặt tính toán.
3. Bất kỳ thay đổi nhỏ nào trong thông điệp đầu vào đều sẽ tạo ra một giá trị băm hoàn toàn khác với xác suất rất cao.

Họ thuật toán SHA bao gồm nhiều phiên bản khác nhau, được phát triển theo từng giai đoạn nhằm nâng cao mức độ bảo mật và khả năng chống lại các cuộc tấn công mật mã học. Các thuật toán chính trong họ SHA bao gồm:

- SHA-1 – tạo ra giá trị băm có độ dài 160 bit
- SHA-224 – tạo ra giá trị băm 224 bit
- SHA-256 – tạo ra giá trị băm 256 bit
- SHA-384 – tạo ra giá trị băm 384 bit
- SHA-512 – tạo ra giá trị băm 512 bit

Trong đó, các thuật toán SHA-224, SHA-256, SHA-384 và SHA-512 thường được gọi chung là SHA-2, được thiết kế như một cải tiến của SHA-1 nhằm tăng cường mức độ an toàn.

Các thuật toán SHA được phát triển bởi Cơ quan An ninh Quốc gia Hoa Kỳ (NSA – National Security Agency) và được tiêu chuẩn hóa bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST – National Institute of Standards and Technology).

Đến năm 2015, NIST đã chính thức công bố SHA-3, là thế hệ thuật toán băm mới được lựa chọn sau một cuộc thi quốc tế kéo dài nhiều năm. SHA-3 bao gồm các biến thể:

- SHA3-224

- SHA3-256
- SHA3-384
- SHA3-512
- SHAKE128
- SHAKE256

Các thuật toán SHA-3 được xây dựng dựa trên cấu trúc sponge construction, khác biệt đáng kể so với cấu trúc Merkle–Damgård được sử dụng trong các phiên bản SHA trước đó.

Tuy nhiên, cần lưu ý rằng SHA-1 hiện nay không còn được coi là an toàn. Vào năm 2005, các nhà nghiên cứu mật mã học đã chỉ ra các phương pháp tấn công lý thuyết nhằm tìm va chạm trong SHA-1 với độ phức tạp thấp hơn dự kiến. Sau đó, nhiều nghiên cứu tiếp theo đã tiếp tục cải thiện các kỹ thuật tấn công này.

Do những hạn chế về bảo mật của SHA-1, cộng đồng nghiên cứu mật mã học đã khuyến nghị chuyển sang sử dụng các thuật toán mạnh hơn như SHA-2 hoặc SHA-3 trong các hệ thống bảo mật hiện đại.

1.3 Kết chương

- Chương 1 đã hệ thống hóa những kiến thức nền tảng về hàm băm mật mã và bức tranh tổng quan về họ thuật toán SHA. Các nội dung trọng tâm bao gồm:
- Bản chất hàm băm: Là công cụ tạo dấu vân tay số thông qua việc ánh xạ dữ liệu tùy ý thành chuỗi nhị phân có độ dài cố định. Một hàm băm đạt chuẩn an toàn phải thỏa mãn tuyệt đối ba tính chất: Kháng tiền ảnh, Kháng tiền ảnh thứ hai và Kháng đụng độ.
- Cơ chế vận hành: Hầu hết các hàm băm hiện đại (ngoại trừ SHA-3) đều dựa trên cấu trúc Merkle–Damgård, thực hiện xử lý dữ liệu theo khối thông qua các vòng lặp hàm nén tuần tự. Điều này đảm bảo tính toàn vẹn và khả năng xác thực nguồn gốc thông điệp (MAC/MDC) trong các hệ thống an ninh mạng.
- Sự tiến hóa của họ SHA:
    - SHA-1: Từng là tiêu chuẩn vàng nhưng hiện nay đã lỗi thời do các lỗ hổng về va chạm (collision).
    - SHA-2 (SHA-256, SHA-512): Là tiêu chuẩn phổ biến nhất hiện nay, cải thiện mạnh mẽ về độ dài băm và khả năng kháng thám mã.
    - SHA-3: Thế hệ mới nhất dựa trên kiến trúc Sponge (Bọt biển), mang lại sự khác biệt hoàn toàn về cấu trúc toán học và khả năng bảo mật hậu lượng tử.

CHƯƠNG 2: GIẢI THUẬT SHA-0

2.1. Giới thiệu về SHA-0

SHA-0 là phiên bản đầu tiên của họ thuật toán SHA và được công bố vào năm 1993 trong tiêu chuẩn FIPS PUB 180 với tên gọi Secure Hash Standard. Thuật toán này được phát triển bởi NSA và được công bố bởi NIST nhằm cung cấp một chuẩn băm mật mã cho các hệ thống của chính phủ Hoa Kỳ.

Tuy nhiên, ngay sau khi được công bố, SHA-0 đã nhanh chóng bị thu hồi do phát hiện các điểm yếu trong thiết kế. Sau đó, một phiên bản sửa đổi của thuật toán đã được phát hành vào năm 1995 trong tiêu chuẩn FIPS PUB 180-1, và phiên bản này được gọi là SHA-1.

SHA-0 là một hàm băm mật mã tạo ra giá trị băm có độ dài 160 bit, được thiết kế dựa trên nguyên lý của thuật toán MD4. Thuật toán này áp dụng mô hình Merkle–Damgård, trong đó một hàm nén (compression function) được lặp lại nhiều lần để xử lý toàn bộ thông điệp đầu vào.

Trong mô hình này, thông điệp đầu vào sẽ được:

1. Đệm (padding) để đảm bảo độ dài phù hợp với kích thước khối xử lý
2. Chia thành các khối dữ liệu có kích thước 512 bit
3. Đưa lần lượt qua một hàm nén (compression function) để cập nhật trạng thái nội bộ

Giả sử thông điệp sau khi đệm được chia thành k khối: M1,M2,.....,Mk

Một giá trị khởi tạo ban đầu Ho (Initial Vector – IV) sẽ được định nghĩa trước. Sau đó, tại mỗi vòng xử lý, trạng thái băm được cập nhật theo công thức:

$$H_{t+1} = h(H_t, M_{t+1})$$

Trong đó:

- H_t là giá trị trạng thái tại vòng t
- M_{t+1} là khối thông điệp tiếp theo
- h là hàm nén

Sau khi xử lý tất cả các khối thông điệp, giá trị Hk sẽ trở thành giá trị băm cuối cùng của thông điệp.

2.2. Thuật toán SHA-0

Hàm nén của SHA-0 được xây dựng dựa trên cấu trúc Davis–Meyer, một cấu trúc phổ biến trong thiết kế các hàm băm dựa trên mật mã khối.

a. Cấu trúc hàm nén

Trong mô hình này, trạng thái băm được cập nhật theo công thức:

$$H_{t+1} = E(H_t, M_{t+1}) \boxplus H_t,$$

b. Cấu trúc bên trong : 80 bước xử lí

Thuật toán SHA-0 sử dụng:

- 80 bước xử lý (steps)
- được chia thành 4 vòng (rounds)
- mỗi vòng gồm 20 bước

Trong quá trình xử lý, năm thanh ghi 32 bit được sử dụng để lưu trạng thái nội bộ: A,B,C,D,E.

c. Mở rộng thông điệp (Message Expansion) :

Khối thông điệp 512 bit ban đầu được chia thành 16 word 32 bit:

$$W_0, W_1, ..., W_{15}$$

Sau đó các từ này được mở rộng thành 80 word theo công thức:

$$W_i = W_{i-16} \oplus W_{i-14} \oplus W_{i-8} \oplus W_{i-3}, \quad 16 \leq i \leq 79$$

Đây là điểm yếu quan trọng của SHA-0 vì:

Không có phép rotate bit => làm giảm độ khuếch tán (diffusion)

d. State update:

Ban đầu :

$$(A_0, B_0, C_0, D_0, E_0) = H_t$$

State Update. First, the chaining variable H_t is divided into 5 32-bit words to fill the 5 registers $(A_0, B_0, C_0, D_0, E_0)$. Then the following transformation is applied 80 times:

$$STEP_{i+1} := \begin{cases} A_{i+1} = (A_i \ll 5) + f_i(B_i, C_i, D_i) + E_i + K_i + W_i, \\ B_{i+1} = A_i, \\ C_{i+1} = B_i \gg 2, \\ D_{i+1} = C_i, \\ E_{i+1} = D_i. \end{cases}$$

Trong đó :

- fi: là hàm Boolean
- Ki: hằng số theo từng round
- <<< : phép xoay trái

e. Feed Forward

Sau 80 bước , trạng thái được cập nhật :

Feed-Forward. The sums modulo 2^{32}: $(A_0 + A_{80})$, $(B_0 + B_{80})$, $(C_0 + C_{80})$, $(D_0 + D_{80})$, $(E_0 + E_{80})$ are concatenated to form the chaining variable H_{t+1}.

Note that all updated registers but A_{i+1} are just rotated copies, so we only need to consider the register A at each step. Thus, we have:

$$A_{i+1} = (A_i \ll 5) + f_i(A_{i-1}, A_{i-2} \gg 2, A_{i-3} \gg 2) + A_{i-4} \gg 2 + K_i + W_i. \quad (1)$$

=> Toàn bộ hệ thống phụ thuộc mạnh vào chuỗi giá trị A

Hàm logic và hằng số

Trong mỗi vòng của thuật toán, một hàm Boolean khác nhau được sử dụng để trộn dữ liệu.

Round	Bước	Hàm Boolean	Hằng số
1	1–20	$(B \wedge C) \vee (\neg B \wedge D)$	0x5A827999
2	21–40	$B \oplus C \oplus D$	0x6ED9EBA1
3	41–60	$(B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$	0x8F1BBCDC
4	61–80	$B \oplus C \oplus D$	0xCA62C1D6

2.5. Demo thực nghiệm phân tích lỗ hổng của SHA-0

Mục tiêu : So sánh cơ chế message expansion giữa SHA-0 và SHA-1 , chứng minh : SHA-0 có cấu trúc tuyến tính → lan truyền sai khác yếu hơn → dễ bị tấn công vi sai

Kết quả :

```
D:\OCR>python -u "d:\OCR\VietCode.py"
SHA-0 - số W[i] khác nhau: 31
SHA-1 - số W[i] khác nhau: 49
```

Hình 1 - Minh hoạ kết quả phân tích lỗ hổng của SHA-0

Thí nghiệm đã chứng minh rằng:

SHA-0 có khả năng khuếch tán yếu do message expansion tuyến tính

Việc thiếu phép xoay bit làm giảm tính ngẫu nhiên

Đây chính là nguyên nhân cốt lõi khiến SHA-0 bị phá vỡ

2.6. Kết chương

Chương 2 đã đi sâu phân tích về SHA-0, thực thể đầu tiên trong họ giải thuật băm an toàn, qua đó rút ra các kết luận quan trọng sau:

1. Kiến trúc nền tảng: SHA-0 đặt nền móng với cấu trúc Merkle–Damgård và hàm nén Davis–Meyer, sử dụng 80 bước xử lý chia thành 4 vòng trên 5 thanh ghi trạng thái (A, B, C, D, E). Đây là mô hình chuẩn mực cho các thế hệ hàm băm kế nhiệm.
2. Lỗ hổng thiết kế cốt lõi: Điểm yếu chí tử của SHA-0 nằm ở cơ chế Message Expansion mang tính tuyến tính thuần túy. Việc thiếu vắng phép xoay bit (Rotate) trong quá trình tạo ra 80 từ thông điệp đã làm suy giảm nghiêm trọng độ khuếch tán (Diffusion).
3. Hệ quả bảo mật: Chính cấu trúc tuyến tính này đã tạo điều kiện cho các kỹ thuật Differential Cryptanalysis phát huy tác dụng. Các nghiên cứu thực nghiệm và Demo đã chứng minh rằng việc tìm kiếm va chạm (Collision) trên SHA-0 có thể thực hiện với chi phí tính toán thấp, khiến thuật toán này không còn giá trị sử dụng trong thực tế.
4. Giá trị tiền đề: Mặc dù bị coi là một thuật toán thất bại về mặt bảo mật, nhưng việc phân tích sai lầm của SHA-0 là bước đệm lý thuyết tối quan trọng. Điều này giúp giải thích tại sao phép xoay bit đơn giản trong SHA-1 sau này lại có thể tạo ra sự khác biệt lớn về khả năng kháng thám mã.

Khép lại Chương 2, chúng ta thấy rõ tầm quan trọng của tính phi tuyến trong mật mã học. Đây là cơ sở để nhóm tiếp tục trình bày về SHA-1 ở chương tiếp theo — một phiên bản cải tiến trực tiếp từ SHA-0 nhằm khắc phục các lỗ hổng vừa nêu.

CHƯƠNG 3: GIẢI THUẬT SHA-1

3.1 Giới thiệu SHA1.

3.1.1. SHA-1 (Secure Hash Algorithm 1) là một hàm băm mật mã được Cơ quan An ninh Quốc gia Hoa Kỳ (NSA) thiết kế và được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ công bố vào năm 1995. Thuật toán này từng được sử dụng rộng rãi để đảm bảo tính toàn vẹn dữ liệu và xác thực chữ ký số, nhưng hiện nay được xem là không còn an toàn.

1.2.Thông tin chính

- Năm công bố: 1995
- Độ dài băm: 160 bit
- Thiết kế bởi: NSA
- Tình trạng bảo mật: Không còn an toàn từ năm 2017
- Kế nhiệm bởi: SHA-2 và SHA-3

1.3. Vai trò lịch sử.

Trong suốt hơn một thập kỷ, SHA-1 là tiêu chuẩn công nghiệp cho bảo mật web (HTTPS, TLS), kiểm tra tính toàn vẹn phần mềm, và chữ ký điện tử như PGP hay Git. Tuy nhiên, các nghiên cứu từ giữa thập niên 2000 đã phát hiện lỗ hổng va chạm khiến hai đầu vào khác nhau có thể tạo cùng một băm.

3.2. Cơ chế hoạt động.

SHA-1 nhận đầu vào là thông điệp có độ dài tùy ý và tạo ra chuỗi băm dài 160 bit (20 byte). Thuật toán hoạt động theo cấu trúc Merkle–Damgård, chia dữ liệu thành các khối 512 bit, sau đó áp dụng các phép toán logic, dịch vòng và cộng modulo để cập nhật giá trị băm trung gian, cuối cùng tạo ra "dấu vân tay" duy nhất cho dữ liệu.

3.2.1. Tiền xử lý (Pre-processing).

- Thông điệp được chia thành các khối 512 bít (tương đương 16 từ, mỗi từ 32 bit). Nếu kích thước thông điệp không là bội số của 512 → nối thêm số bít thiếu;
- Thêm các bit đệm (Padding bits):
  - Thêm một bit "1" vào ngay sau thông điệp gốc.
  - Tiếp tục thêm các bit "0" cho đến khi độ dài thông điệp đạt mức 448 bit (tính theo modulo 512).
  - Lấy độ dài gốc của thông điệp (L) biểu diễn dưới dạng số nhị phân 64 bit.
  - Công thức: L+1+K+64 mod (trong đó L là độ dài gốc, K là số bit 0 cần thêm, 64 bit biểu diễn độ dài thông điệp L).
- Chia khối dữ liệu (Parsing):
  - Chia toàn bộ thông điệp đã đệm thành các khối M1, m2, m3, m4, …, mỗi khối có độ dài đúng 512 bit.

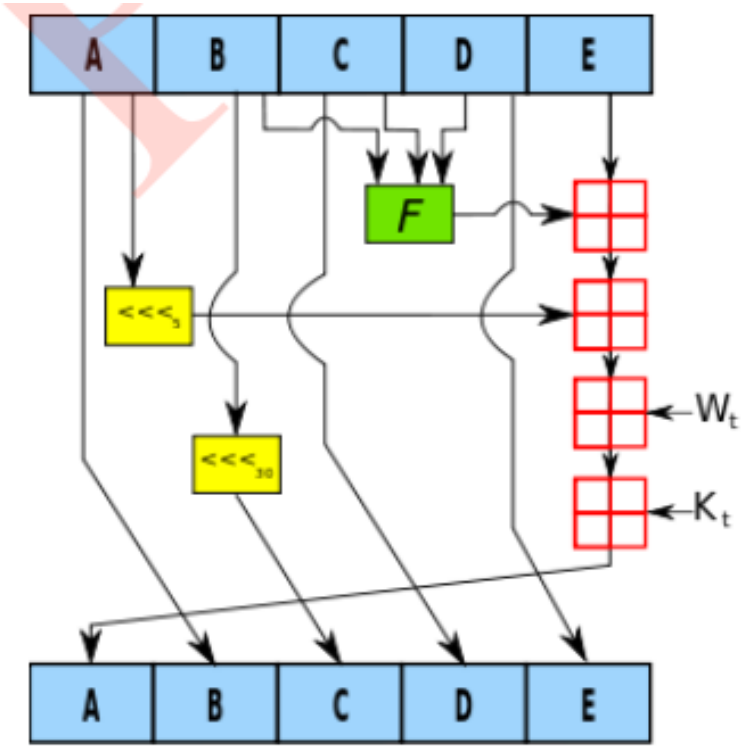
3.2.2. Các vòng lặp xử lý:

- Phần xử lý chính của SHA1 làm việc trên state 160 bit, chia thành 5 từ 32 bit (A, B, C, D, E):
  - Các từ A, B, C, D, E được khởi trị bằng một hằng cố định;
  - Từng phần 32 bit của các khối đầu vào 512 bit được đưa dần vào để thay đổi state;

- Mở rộng thông điệp: Chia khối 512 bit thành 16 từ (word) 32-bit (W0 đến W15). Sau đó mở rộng chúng thành 80 từ (W0 đến W79) bằng các phép dịch bit và XOR tuyến tính như sau:
  - Các từ còn lại W16→W 79 được tính bằng công thức:

$$W_t = \mathrm{ROTL}^1\left(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}\right)$$

- Quá trình xử lý gồm 80 vòng, mỗi vòng gồm các thao tác: add, and, or, xor, rotate, mod.
- Lưu đồ xử lý 1 vòng của SHA1:



Hình 2 - Lưu đồ xử lý 1 vòng của SHA1

- A, B, C, D, E: các từ 32 bit được khởi trị bằng 1 hằng cố định:
- Khối Wt: khối 32 bit thông điệp đầu vào dược lấy từ các khối Wi (0<i<80) được xử lí bên trên.
- Kt là hằng 32 bit khác nhau cho mỗi vòng.
- <<<n: là thao tác dịch trái n bit:

- ⊞ biểu diễn cộng modulo 32 bit.
- Xử lý bởi hàm phi tuyến tính F (có nhiều dạng hàm khác nhau):

$$F_t(B,C,D) = \begin{cases} (B \wedge C) \vee ((\neg B \wedge D) & \text{nếu } 0 \le t \le 19 \\ B \oplus C \oplus D & \text{nếu } 20 \le t \le 39 \\ (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) & \text{nếu } 40 \le t \le 59 \\ B \oplus C \oplus D & \text{nếu } 60 \le t \le 79 \end{cases}$$

Sau mỗi vòng lặp các khối được cập nhật như sau:

$$\text{STEP}_{i+1} := \begin{cases} A_{i+1} = (A_i \lll 5) + f_i(B_i, C_i, D_i) + E_i + K_i + W_i, \\ B_{i+1} = A_i, \\ C_{i+1} = B_i \lll 30, \\ D_{i+1} = C_i, \\ E_{i+1} = D_i. \end{cases}$$

3.2.3. Output:

- Sau khi xử lý xong tất cả các khối, giá trị cuối cùng của các biến A, B, C, D, E được nối lại với nhau để tạo thành chuỗi 160 bit.
- Kết quả thường được biểu diễn dưới dạng chuỗi thập lục phân (hexadecimal) dài 40 ký tự.

3.3. SHA1 và Strict Avalanche Criterion (SAC)

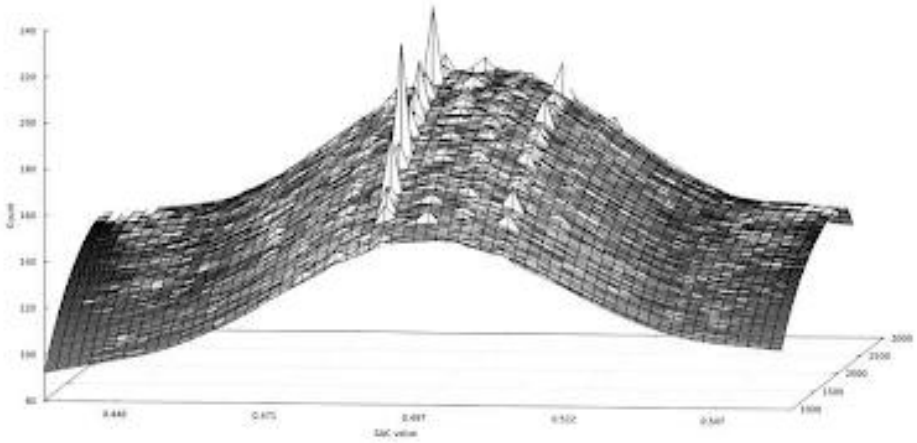
3.3.1 Định nghĩa.

Strict Avalanche Criterion (SAC) là một tiêu chí quan trọng dùng để đánh giá mức độ khuếch tán của một hàm mật mã. Theo định nghĩa, một hàm mật mã F thỏa mãn SAC nếu khi thay đổi một bit của dữ liệu đầu vào, thì mỗi bit của đầu ra có xác suất khoảng 1/2 bị thay đổi. Nói cách khác, sự thay đổi rất nhỏ ở đầu vào phải gây ra sự thay đổi lớn và gần như ngẫu nhiên ở đầu ra. Điều này thường được đo bằng khoảng cách Hamming giữa hai giá trị băm thu được khi chỉ thay đổi một bit của input.

Ý nghĩa: SAC thể hiện hiệu ứng avalanche (avalanche effect) trong các thuật toán mật mã, tức là một thay đổi nhỏ ở đầu vào sẽ lan truyền và ảnh hưởng đến nhiều bit của đầu ra. Điều này giúp đảm bảo hai tính chất quan trọng của hệ mật mã là confusion (làm phức tạp mối quan hệ giữa input và output) và diffusion (khuếch tán sự thay đổi của dữ

liệu). Nhờ đó, hàm băm trở nên khó dự đoán hơn và tăng khả năng chống lại các dạng tấn công như tìm preimage hoặc phân tích sự khác biệt giữa các đầu vào. Trong nghiên cứu đối với SHA-1, kết quả cho thấy các bit đầu ra nhanh chóng đạt gần giá trị SAC lý tưởng sau một số vòng xử lý, chứng tỏ thuật toán có khả năng khuếch tán thay đổi rất tốt.

3.3.2 Đồ thị 3D biểu diễn kết quả kiểm tra tính chất Strict Avalanche Criterion (SAC) của.



Hình 3 - Đồ thị 3D biểu diễn kết quả kiểm tra tính chất Strict Avalanche Criterion

Trục:

- X - giá trị SAC: thể hiện xác suất thay đổi của các bit đầu ra khi một bit đầu vào bị thay đổi. Đối với một hàm băm lý tưởng, giá trị này phải xấp xỉ 0.5.
- Y - Trục Count: Số lượng các cặp bit thỏa mãn giá trị SAC tương ứng
- Z: số lượng bit

Hình dáng đồ thị:

- Một hàm băm tốt sẽ có đồ thị dạng hình chuông (Gaussian) tập trung cao độ tại điểm 0.5 (như đỉnh núi trong ảnh bạn gửi).
- Các gai nhọn nhô lên cao bất thường hoặc đồ thị bị lệch cho thấy thuật toán có điểm yếu, khiến hacker có thể dự đoán được quy luật thay đổi của các bit.

3.4. Ưu và nhược điểm.

3.4.1 Ưu điểm

+ Tốc độ xử lý nhanh

So với các thuật toán hiện đại như SHA-256 hay SHA-3, SHA-1 có cấu trúc đơn giản hơn và ít vòng lặp hơn. Điều này giúp việc tính toán mã băm diễn ra rất nhanh, tiết kiệm tài nguyên CPU.

+ Tiết kiệm không gian lưu trữ

Mã băm của SHA-1 chỉ dài 160 bit (40 ký tự hexa). Nó ngắn hơn đáng kể so với SHA-256 (256 bit) hay SHA-512. Trong các hệ thống lưu trữ khổng lồ, việc dùng mã băm ngắn giúp tiết kiệm bộ nhớ và băng thông truyền tải.

+. Tính tương thích cực cao

Vì ra đời từ sớm (1995), SHA-1 được hỗ trợ bởi hầu hết các thiết bị cũ, hệ điều hành cổ điển và các thư viện lập trình. Trong các hệ thống cũ (Legacy systems) không thể nâng cấp phần cứng, SHA-1 vẫn là lựa chọn duy nhất hoạt động được.

+ Hiệu quả trong kiểm tra toàn vẹn dữ liệu (không bảo mật)

Trong những trường hợp bạn chỉ cần kiểm tra xem file tải về có bị lỗi đường truyền hay không (Checksum), mà không sợ có ai đó cố tình "giả mạo" file, thì SHA-1 vẫn rất hiệu quả và kinh tế.

+ Dễ triển khai trên phần cứng yếu

Với các thiết bị IoT hoặc vi điều khiển có cấu hình thấp, việc chạy SHA-1 ít tốn RAM và năng lượng hơn các thuật toán phức tạp đời mới.

3.4.2. Nhược điểm.

+ Lỗi đụng độ (Collision)

Đây là lỗ hổng nghiêm trọng nhất. Theo lý thuyết, mỗi thông điệp khác nhau phải cho ra một mã băm duy nhất. Tuy nhiên, SHA-1 đã bị chứng minh là có thể tạo ra cùng một mã băm cho hai thông điệp khác nhau.

- Sự kiện Google (2017): Các chuyên gia tại Google đã tạo ra hai tệp PDF có nội dung khác biệt hoàn toàn nhưng lại có cùng mã SHA-1. Điều này cho phép kẻ tấn công thay thế một tài liệu hợp pháp (như hợp đồng) bằng một tài liệu giả mạo mà chữ ký số vẫn báo là "hợp lệ".

+. Độ dài mã băm quá ngắn

SHA-1 chỉ tạo ra mã băm dài 160 bit. Với sự phát triển của phần cứng (GPU và điện toán đám mây), việc dùng phương pháp "vét cạn" (brute-force) để tìm ra các chuỗi trùng mã băm trở nên rẻ và nhanh hơn rất nhiều so với trước đây.

+ Cấu trúc toán học lỗi thời

Thuật toán SHA-1 dựa trên cấu trúc Merkle-Damgård, vốn có những điểm yếu cố hữu về mặt toán học mà các máy tính hiện đại có thể khai thác.

3.5 Demo.

3.5.1 Demo thực thi giải thuật mã hóa SHA1.

- Kết quả demo và kết quả khi sử dụng thư viện mã hóa SHA1: Code Demo

```
PS D:\atbm> & C:\Users\CPS\AppData\Local\Programs\Python\Python312\python.
Chuỗi: hello
SHA1 thủ công: aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d
SHA1 dùng hàm: aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d
```

Hình 4 - Kết quả demo thực thi giải thuật SHA1

3.5.2 Demo va chạm trên SHA-1:

Tấn công va chạm nhằm mục tiêu tìm hai thông điệp khác nhau nhưng cho cùng một giá trị băm. Đối với SHA-1, mặc dù độ dài đầu ra là 160 bit, nhưng theo nguyên lý ngày sinh nhật, kẻ tấn công chỉ cần khoảng 2^80 phép thử để tìm va chạm với xác suất

đáng kể. Ngoài ra, các nghiên cứu thực tế đã chỉ ra rằng độ phức tạp này có thể giảm xuống nhờ khai thác điểm yếu trong cấu trúc toán học của SHA-1, tiêu biểu là tấn công **SHAttered attack**. Do đó, SHA-1 hiện nay không còn được xem là an toàn cho các ứng dụng yêu cầu tính toàn vẹn và xác thực dữ liệu. CodeDemo

```
SHA1 file 1: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a
SHA1 file 2: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a
>>> VA CHẠM SHA-1!
```

Hình 5 - Kết quả demo tấn công va chạm mã SHA

3.6 Kết Chương

Chương này đã trình bày một cách tổng quan và chi tiết về giải thuật băm SHA-1, từ lịch sử hình thành, cấu trúc, cho đến cơ chế hoạt động cụ thể. Thông qua việc phân tích các bước tiền xử lý, mở rộng thông điệp và các vòng lặp xử lý, có thể thấy SHA-1 được thiết kế với mục tiêu đảm bảo tính toàn vẹn dữ liệu thông qua các phép toán logic và số học tương đối hiệu quả. Bên cạnh đó, việc đánh giá theo tiêu chí Strict Avalanche Criterion (SAC) cho thấy thuật toán có khả năng khuếch tán tốt, đảm bảo sự thay đổi nhỏ ở đầu vào có thể gây ảnh hưởng lớn đến đầu ra.

Tuy nhiên, cùng với sự phát triển của công nghệ và các phương pháp phân tích mật mã hiện đại, SHA-1 đã bộc lộ nhiều điểm yếu nghiêm trọng, đặc biệt là khả năng xảy ra va chạm. Các minh chứng thực tế như cuộc tấn công SHAttered đã khẳng định rằng SHA-1 không còn đáp ứng được các yêu cầu bảo mật trong những hệ thống hiện đại. Dù vậy, nhờ ưu điểm về tốc độ và tính tương thích, SHA-1 vẫn có thể được sử dụng trong một số trường hợp không yêu cầu mức độ bảo mật cao, chẳng hạn như kiểm tra tính toàn vẹn dữ liệu đơn thuần.

Tóm lại, việc nghiên cứu SHA-1 không chỉ giúp hiểu rõ nguyên lý hoạt động của các hàm băm mật mã mà còn là cơ sở quan trọng để tiếp cận và đánh giá các thuật toán băm tiên tiến hơn như SHA-2 và SHA-3 trong các hệ thống bảo mật hiện nay.

CHƯƠNG 4: GIẢI THUẬT SHA-2

4.1. Lịch sử và tiêu chuẩn

-Do National Security Agency thiết kế

-Chuẩn hóa bởi National Institute of Standards and Technology

-SHA-2 kế thừa SHA-1, tuy nhiên, có những thay đổi đáng kể so với SHA-1

-Họ SHA-2 gồm 6 hàm băm với giá trị băm khác nhau:

+) SHA-224: 224 bit. Đây là phiên bản cắt gọn của SHA-256 tính toán với các giá trị ban đầu khác nhau.

+) SHA-256: 256 bit. Đây là thuật toán SHA-2 tiêu chuẩn với kích thước băm 256 bit.

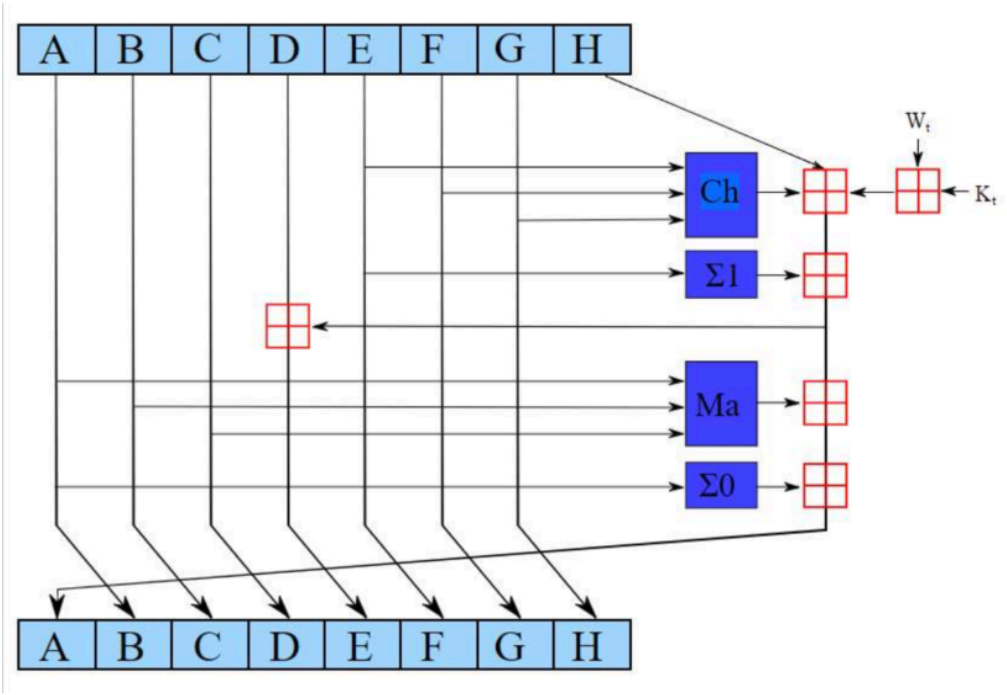
+) SHA-384: 384 bit. Đây là phiên bản cắt gọn của SHA-512 tính toán với các giá trị ban đầu khác nhau.

+) SHA-512: 512 bit. Đây là thuật toán SHA-2 tiêu chuẩn với kích thước băm 512 bit.

+) SHA-512/224: Đây là phiên bản cắt gọn của SHA-512.

+) SHA-512/256: Đây cũng là một phiên bản cắt gọn của SHA-512.

Note: SHA-256 được sử dụng rộng rãi, đặc biệt là bởi các cơ quan chính phủ Hoa Kỳ để bảo mật dữ liệu nhạy cảm của họ. Mạnh hơn nhiều so với SHA-1, nó bao gồm thuật toán băm an toàn hơn nhiều. SHA-256, cũng được sử dụng trong các giao dịch Bitcoin.



Hình 6 - Lưu đồ một vòng lặp SHA-2

$$Ch(E,F,G) = (E \wedge F) \oplus (\neg E \wedge G)$$
$$Ma(A,B,C) = (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C)$$
$$\sum_0 (A) = (A >>> 2) \oplus (A >>> 13) \oplus (A >>> 22)$$
$$\sum_1 (E) = (E >>> 6) \oplus (E >>> 11) \oplus (E >>> 25)$$

Hình 7 - Công thức biểu diễn nguyên lý hoạt động của SHA-2

4.2. Tìm hiểu cấu trúc SHA-256:

4.2.1. Khái niệm:

- Là thuật toán bảo mật 256bit

- Dùng để tạo các hàm băm không thể đảo ngược và duy nhất

- Số lượng hàm băm càng lớn thì xác suất để 2 giá trị có thể tạo ra 1 giá trị hàm băm càng nhỏ

⇒ Phù hợp để xác nhận mật khẩu, chống giả mạo, chữ ký số,...

VD nổi bật: Hệ thống Bitcoin,...

4.2.2. Tính chất của hàm băm SHA-256:

*** Tính xác định:**

- Ý nghĩa: Cùng một dữ liệu đầu vào sẽ luôn tạo ra cùng một giá trị hash.

Giải thích:

- Nếu một thông điệp được băm nhiều lần bằng cùng một thuật toán SHA-2, kết quả hash sẽ không thay đổi.

*** Độ dài đầu ra cố định:**

- Ý nghĩa:

Kết quả hash luôn có độ dài cố định, bất kể dữ liệu đầu vào dài hay ngắn.

*** Khả năng chống tìm ngược đầu vào:**

- Ý nghĩa:

Từ một giá trị hash đã biết, rất khó để tìm ra dữ liệu ban đầu tạo ra hash đó.

- Giải thích:

Nếu biết:

H = SHA-256(M)

thì việc tìm lại M gần như không khả thi.

*** Khả năng chống tìm dữ liệu thứ hai có cùng hash:**

- Ý nghĩa: Nếu đã biết một dữ liệu M1, thì rất khó tìm một dữ liệu khác M2 (M2 ≠ M1) sao cho:

SHA-256(M1) = SHA-256(M2)

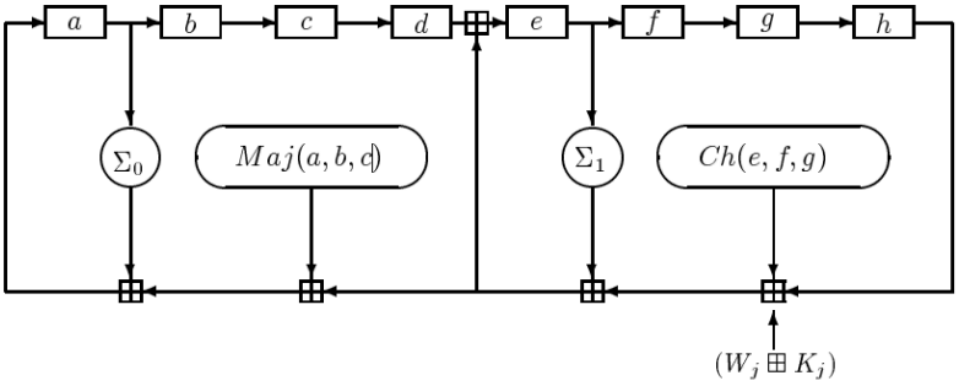
*** Khả năng chống va chạm:**

- Ý nghĩa: Rất khó tìm hai dữ liệu bất kỳ khác nhau nhưng tạo ra cùng một giá trị hash.

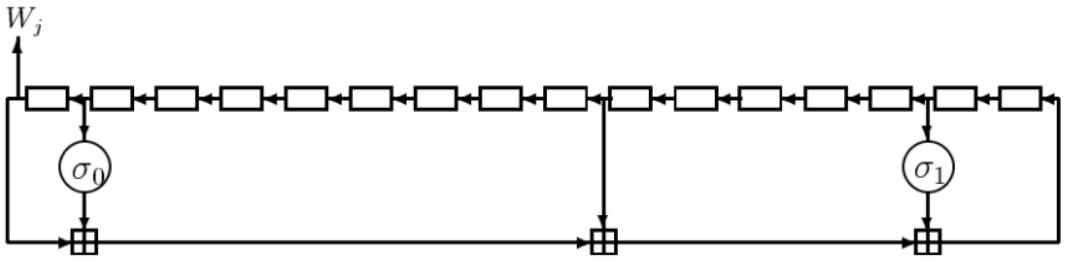
Ví dụ: M1 ≠ M2 nhưng SHA-256(M1) = SHA-256(M2)

*** Hiệu ứng tuyết lở:**

- Ý nghĩa: Chỉ cần thay đổi một ký tự nhỏ trong dữ liệu đầu vào, giá trị hash sẽ thay đổi hoàn toàn.



Hình 8 - Bước nội bộ thứ j của hàm nén SHA-256 C



Hình 9 - Lịch trình thông điệp SHA-256

4.2.3. Các bước thực hiện SHA-256:

B1: Khởi tạo các giá trị băm:

(32 bit đầu tiên của các phần phân đoạn của hình vuông gốc của 8 số nguyên tố đầu tiên 2..19):

h0: = 0x6a09e667 h1: = 0xbb67ae85 h2: = 0x3c6ef372 h3: = 0xa54ff53a

h4: = 0x510e527f h5: = 0x9b05688c h6: = 0x1f83d9ab h7: = 0x5be0cd19

B2: Khởi tạo mảng các hằng số tròn:

(32 bit đầu tiên của các phần phân đoạn của các phần tử lập phương của 64 số nguyên tố đầu tiên 2..311):

0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1,

0x923f82a4, 0xab1c5ed5,

0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe,

0x9bdc06a7, 0xc19bf174,

0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa,

0x5cb0a9dc, 0x76f988da,

0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147,

0x06ca6351, 0x14292967,

0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb,

0x81c2c92e,0x92722c85,0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,

0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,

0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2

B3: Quá trình xử lý

Bắt đầu với thông điệp ban đầu của độ dài L bit nối thêm một bit '1' vào bit

K '0', trong đó K là số tối thiểu> = 0, ví dụ: rằng L + 1 + K + 64 là một bội số của 512

phụ L như là một số nguyên lớn-64-endian, làm cho tổng chiều dài sau xử lý một bội số của 512 bit

B4: Xử lý thông báo trong các khối 512 bit kế tiếp:

chia nhỏ tin nhắn thành các khối 512 bit cho mỗi đoạn tạo một mảng lịch thông điệp 64

mục nhập w [0..63] của các từ 32 bit (Các giá trị ban đầu trong w [0 63] không thành vấn đề, quá nhiều hiện thực không cho họ ở đây) sao chép đoạn vào 16 từ đầu tiên w [0..15]

của mảng lịch thông báo Mở rộng 16 từ đầu tiên vào 48 từ còn lại w [16..63] mảng thông điệp: cho i từ 16 đến 63

for i from 16 to 63

s0 := (w[i-15] rightrotate 7) xor (w[i-15] rightrotate 18) xor (w[i-15] rightshift 3)

s1 := (w[i-2] rightrotate 17) xor (w[i-2] rightrotate 19) xor (w[i-2] rightshift 10)

w[i] := w[i-16] + s0 + w[i-7] + s1

B5: Khởi tạo biến làm việc với giá trị băm hiện tại:

a: = h0 b: = h1 c: = h2 d: = h3	e: = h4 f: = h5 g: = h6 h: = h7

B6: Nén vòng lặp chính: cho i từ 0 đến 63

for i from 0 to 63

S1 := (e rightrotate 6) xor (e rightrotate 11) xor (e rightrotate 25)

ch := (e and f) xor ((not e) and g)

temp1 := h + S1 + ch + k[i] + w[i]

S0 := (a rightrotate 2) xor (a rightrotate 13) xor (a rightrotate 22)

maj := (a and b) xor (a and c) xor (b and c) temp2 := S0 + maj

h: = g g: = f f: = e e: = d + temp1	d: = c c: = b b: = a a: = temp1 + temp2

B7: Thêm đoạn nén vào băm hiện tại giá trị:

h0: = h0 + a h1: = h1 + b h2: = h2 + c h3: = h3 + d	h4: = h4 + e h5: = h5 + f h6: = h6 + g h7: = h7 + h

B8: Tạo ra giá trị băm cuối cùng (big-endian):

digest := hash := h0 append h1 append h2 append h3 append h4 append h5 append h6 append h7

4.3. Demo thực nghiệm tìm Collision (Tìm va chạm) của SHA-256:

Link: https://github.com/nvbangg/demo-sha/blob/main/demo/sha2.py

4.3.1. Kết quả demo:

- Thực nghiệm này nhằm mô phỏng bài toán tìm va chạm (Collision) của hàm băm SHA-256 trong môi trường kiểm soát. Do không gian mẫu 2^{256} nguyên bản là quá lớn để tính toán, chúng ta sử dụng kỹ thuật cắt ngắn mã băm (Truncated Hash) xuống các mức độ khó thấp hơn (16-bit, 24-bit)

- Mục tiêu chính là quan sát số lần thử cần thiết để tìm thấy hai dữ liệu khác nhau có cùng giá trị băm, từ đó kiểm chứng Nghịch lý ngày sinh (Birthday Paradox). Kết quả thực nghiệm giúp chứng minh rằng độ an toàn thực tế của một hàm băm chỉ bằng một nửa độ dài bit của nó $2^{n/2}$, khẳng định tầm quan trọng của việc duy trì độ dài 256-bit để đảm bảo tính toàn vẹn dữ liệu tuyệt đối trong thực tế.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
[Running] python -u "e:\Downloads\test.py"
Độ khó (Bit)     | Số lần thử       | Thời gian (s)
-------------------------------------------------
12               | 84               | 0.0006
16               | 251              | 0.0002
20               | 1,877            | 0.0024
24               | 8,864            | 0.0094

[Done] exited with code=0 in 0.368 seconds
```

Hình 10 - Kết quả demo thử nghiệm tìm va chạm

4.3.2. Kết luận rút ra:

- Kết quả cho thấy khi độ dài hash tăng, số lần thử để tìm collision tăng rất nhanh.

  => Tính bảo mật cao của các hàm hash hiện đại như SHA-256

- Hiện tượng này liên quan đến birthday paradox, số lần thử là: $2^{n/2}$

- Bài thử nghiệm chỉ nhằm mục đích minh họa nguyên lý hoạt động của collision trong mật mã học

4.4 Kết chương

Chương 4 đã phân tích chi tiết về **SHA-2**, giải thuật băm đóng vai trò nền tảng trong bảo mật hiện đại. Qua đó, có thể tổng kết các nội dung trọng tâm sau:

- **Cấu trúc tối ưu:** SHA-2 vượt xa các thế hệ tiền nhiệm nhờ thiết kế đa dạng (từ SHA-224 đến SHA-512), cho phép tùy biến linh hoạt và đạt hiệu suất cao trên cả hệ thống 32-bit và 64-bit.
- **Cơ chế kháng thám mã mạnh:** Sức mạnh của SHA-256 nằm ở quy trình nén 64 vòng lặp kết hợp các hàm phi tuyến (Ch, Maj, Σ). Đặc biệt, cơ chế mở rộng thông điệp đã loại bỏ tính tuyến tính của SHA-0, giúp chống lại hiệu quả các cuộc tấn công vi sai.
- **Tính bảo mật tuyệt đối:** Với khả năng kháng va chạm (Collision Resistance) và kháng tiền ảnh (Pre-image Resistance) cực cao, SHA-256 hiện là tiêu chuẩn vàng cho các hệ thống yêu cầu độ tin cậy tuyệt đối như Blockchain, Bitcoin và chữ ký số chính phủ.
- **Giá trị thực tiễn:** Việc khắc phục thành công các lỗ hổng của SHA-0/SHA-1 đã giúp SHA-2 duy trì vị thế kiên cố trong tiêu chuẩn FIPS PUB 180-4, trở thành thước đo an ninh cho các hệ thống mã hóa dữ liệu nhạy cảm toàn cầu.

Tổng kết lại, việc nắm vững cấu trúc SHA-2 không chỉ giúp khẳng định độ an toàn của các hệ thống hiện tại mà còn là tiền đề quan trọng để tiếp cận các thế hệ hàm băm tiên tiến như SHA-3.

CHƯƠNG 5: GIẢI THUẬT SHA-3

5.1. Giới thiệu SHA-3

5.1.1. Lịch sử ra đời

Mặc dù SHA-2 vẫn đang an toàn, nhưng do SHA-2 sử dụng chung cấu trúc Merkle-Damgård với các giải thuật đã bị bẻ gãy như SHA-1 và MD5 nên đã đặt ra yêu cầu cấp thiết về một phương án dự phòng chiến lược. Vào năm 2007 Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) khởi xướng cuộc thi thiết kế hàm băm công khai nhằm tìm kiếm một tiêu chuẩn mới có cấu trúc khác biệt hoàn toàn với các thế hệ trước. Sau nhiều vòng đánh giá khắt khe, thuật toán Keccak đã được lựa chọn và chính thức chuẩn hóa thành SHA-3 (Secure Hash Algorithm 3) theo tiêu chuẩn FIPS PUB 202 vào năm 2015.

5.1.2. Các phiên bản

SHA-3 bao gồm 6 phiên bản

- SHA-3 với độ dài đầu ra cố định: gồm SHA3-224, SHA3-256, SHA3-384, SHA3-512. Các chỉ số phiên bản trên tương ứng với độ dài bit đầu ra. Ví dụ, SHA3-256 tạo ra mã băm 256 bit
- SHA-3 với độ dài đầu ra tùy chỉnh, gọi là Extendable-Output Functions (XOFs): gồm SHAKE128 và SHAKE256

5.1.3. Ưu điểm và hạn chế

- Độ an toàn cao: SHA-3 sử dụng cấu trúc "Bọt biển" khác biệt, giúp không bị ảnh hưởng bởi cấu trúc Merkle–Damgård truyền thống. Có thể chống lại kiểu tấn công mở rộng độ dài (Length Extension Attack) mà không cần cài đặt thêm lớp bảo vệ phức tạp.
- Linh hoạt và tối ưu: Cho phép tạo hash với độ dài tùy ý, tối ưu trên các mạch tích hợp chuyên dụng như ASIC và FPGA giúp tăng hiệu suất, tiết kiệm năng lượng trong các thiết bị nhỏ gọn
- Hạn chế: Mặc dù tối ưu cho các thiết bị phần cứng chuyên dụng, nhưng trên các bộ vi xử lý (CPU) thông thường của máy tính và điện thoại, SHA-3 thường có tốc độ thực thi chậm hơn so với SHA-2

5.1.4 Ứng dụng

- Ethereum: Keccak-256 (phiên bản tiền chuẩn của SHA-3) được dùng trong Ethereum để giao dịch, ví, smart contract, lưu trữ và kiểm tra dữ liệu,...
- IoT và Hệ thống nhúng: Được dùng rộng rãi trong chip bảo mật và thiết bị thông minh (camera, gia dụng) nhờ hiệu suất tối ưu, tiết kiệm năng lượng trên mạch chuyên dụng

- Xác thực: Được dùng trong các tiêu chuẩn chữ ký số hiện đại (ví dụ Ed448 dùng SHAKE-256)
- Thay thế SHA-2: Trong các hệ thống hiện đại quan trọng.

5.2. Kiến trúc giải thuật

5.2.1. Giới thiệu về State

State là mảng 3 chiều dùng làm bộ nhớ tạm trong quá trình tính toán. Giá trị khởi tạo ban đầu đều bằng 0. State gồm 2 phần: Rate (r) và Capacity (c).

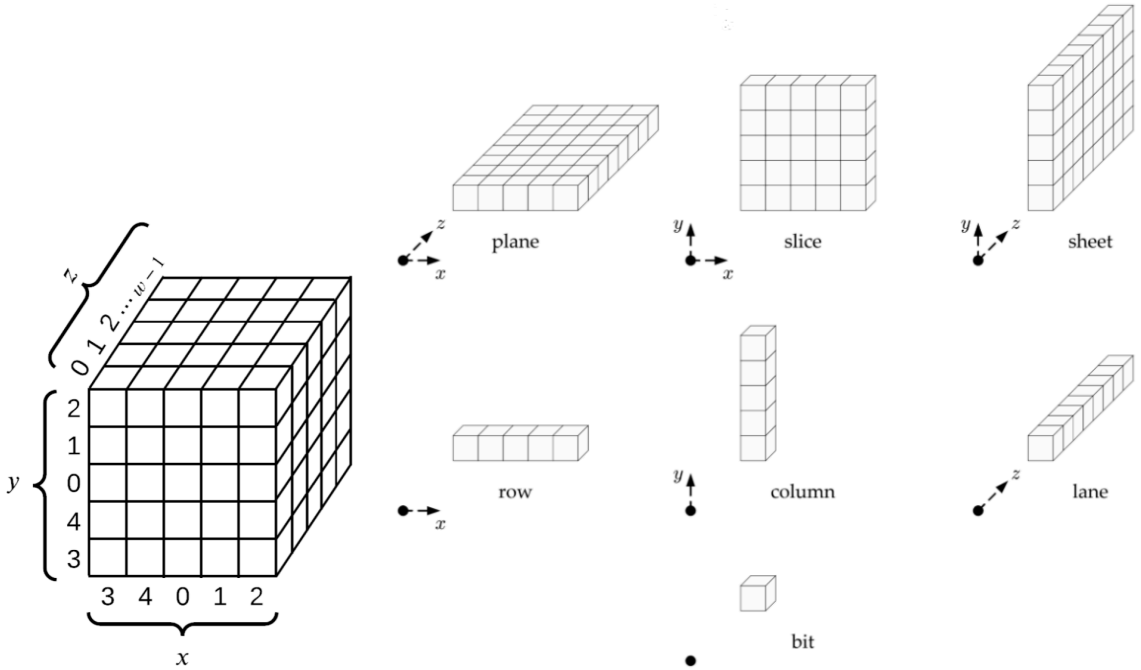
- Rate (r) là phần State có thể tương tác trực tiếp với input.
- Capacity (c) là phần State nội bộ, không tiếp xúc trực tiếp với input.

Trong SHA-3, Tổng r + c luôn bằng 1600 bit

Bảng 1. Giá trị r, c, d của SHA-3

SHA-3 variant	r (rate)	c (capacity)	d (số bit độ dài đầu ra)
sha3-224	1152 bits	448 bits	224 bits
sha3-256	1088 bits	512 bits	256 bits
sha3-384	832 bits	768 bits	384 bits
sha3-512	576 bits	1024 bits	512 bits
shake128	1344 bits	256 bits	tùy ý
shake256	1088 bits	512 bits	tùy ý

Lưu ý State được biểu diễn theo hệ tọa độ tâm (-2,-1,0,1,2 (mod 5) → 3,4,0,1,2). Thứ tự ánh xạ bit vào State được thực hiện theo trình tự ưu tiên các trục: z → x → y .

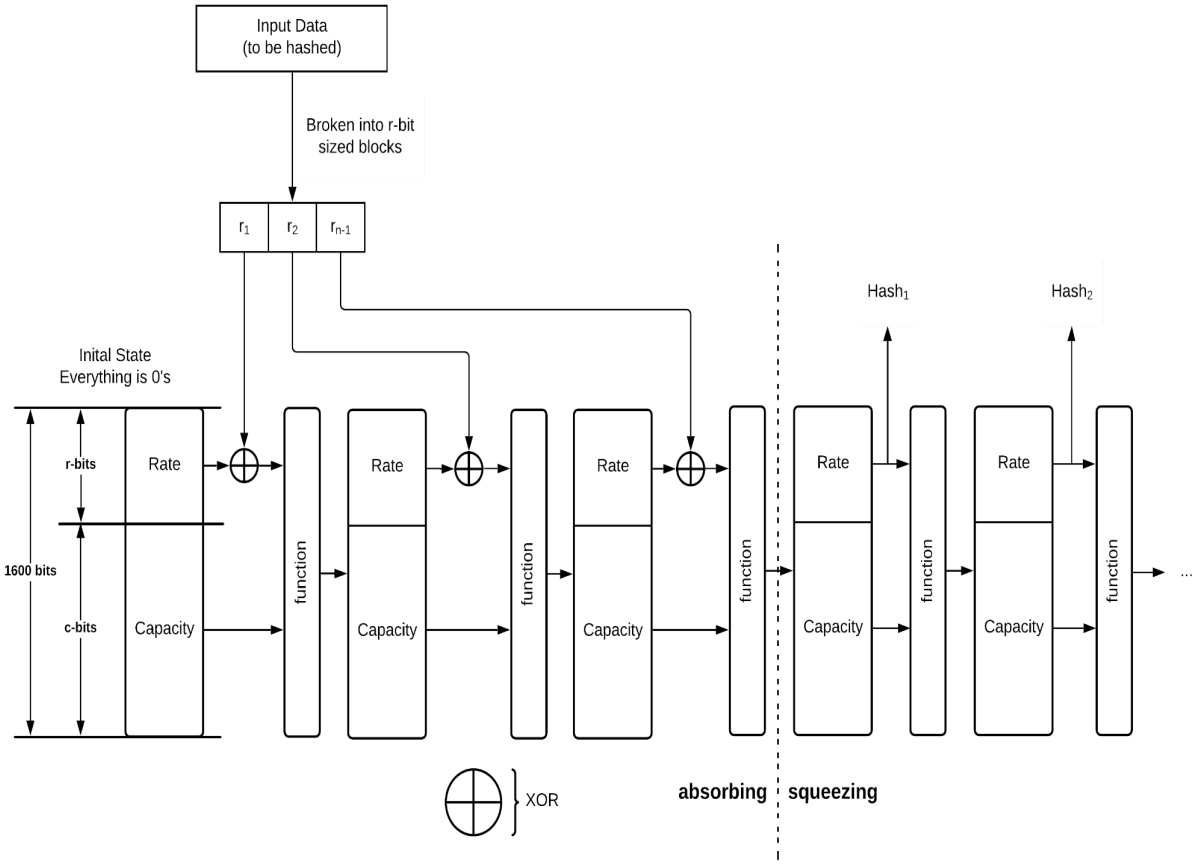


Hình 11 - Mô tả các thành phần của state

5.2.2. Luồng hoạt động cấu trúc “Bọt biển”

Thuật toán SHA-3 sử dụng một cấu trúc hoàn toàn khác biệt so với các thế hệ SHA trước đó, gọi là cấu trúc "Bọt biển" (Sponge Construction):
Cấu trúc bọt biển của SHA-3 hoạt động bằng cách:

1. Chia dữ liệu đầu vào thành các block r bit (r1, r2,...,r_{n-1}); State khởi tạo đều là 0
2. Phần Rate được thực hiện phép XOR với khối r1 đầu tiên.
3. Rate kết quả và Capacity kết hợp lại được nạp vào một function gồm nhiều vòng biến đổi
4. r bit đầu tiên của đầu ra function là Rate và c bit còn lại là Capacity.
5. Phần Rate được XOR với r2 và kết hợp với Capacity nạp vào một function khác.
6. Quá trình này được thực hiện cho đến khi không còn khối dữ liệu nào dư lại.
7. Ở khối dữ liệu cuối cùng, mã băm (hash) được lấy từ r bit đầu ra của function.
8. Nếu cần thêm các bit đầu ra, phần rate và capacity ở trên sẽ được nạp vào function mà không cần nhập thêm bất kỳ dữ liệu nào để tiếp tục lấy r bit từ đầu ra đến khi đủ



Hình 12- Sơ đồ hàm Bọt biển

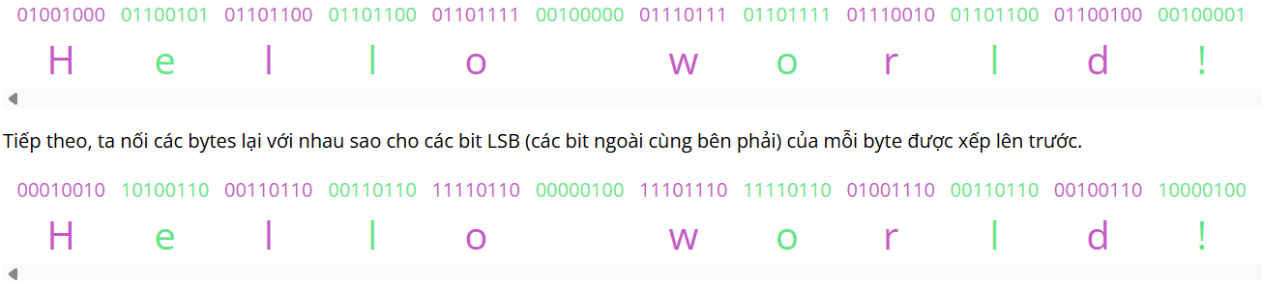
5.3. Giải thuật băm SHA3

5.3.1. Thu được mảng bit

Quy trình thực hiện:

1. Chuẩn hóa dữ liệu đầu vào thành chuỗi byte

2. Đảo ngược thứ tự các bit trong mỗi byte để chuyển từ cách biểu diễn MSB-first (bit trọng số cao nhất đứng đầu) sang LSB-first (bit trọng số thấp nhất đứng đầu)



Hình 13- Minh họa thu mảng bit của string "Hello world!"

5.3.2. Thêm padding và chia thành các block

Padding là quá trình thêm các bit bổ sung vào mảng bit để mảng bit thỏa mãn một điều kiện nhất định. Đối với SHA-3, quy trình tổng quát như sau:

bits + (01 hoặc 1111 nếu là SHAKE) + 1 + n số 0 + 1

Số lượng số 0 thêm vào sao cho độ dài của chuỗi trên là bội số của giá trị r tương ứng phiên bản SHA-3 đã lựa chọn

Sau khi padding hoàn tất, ta thu được một mảng bit có độ dài chia hết cho r. Tiếp theo, mảng bit này được chia thành các block có độ dài r bit.

5.3.3. Giai đoạn "Hấp thụ"

Sau khi padding và chia thành các block, khởi tạo tất cả bit của State đều bằng 0.

Quy trình tổng quát giai đoạn "Hấp thụ" (Absorbing Phase) như sau:

For mỗi input block:

- XOR(state, input block)
- For 24 vòng:
  - θ heta(state)
  - ρ rho(state)
  - π pi(state)
  - χ chi(state)
  - ι iota(state, round)

5.3.3.1. Bước XOR dữ liệu

Đầu vào: Khối dữ liệu hiện tại r_i (độ dài r bit) và State hiện tại S (1600 bit).

Quy trình chi tiết:

1. Chia State thành 2 phần: Rate (r bit đầu tiên) và Capacity (c bit còn lại).
2. Input block r_i được XOR với phần Rate của State.
3. Giữ nguyên phần Capacity

Kết quả: Một State mới được hình thành, sẵn sàng đi vào hàm hoán vị f

5.3.3.2. Bước θ (theta)

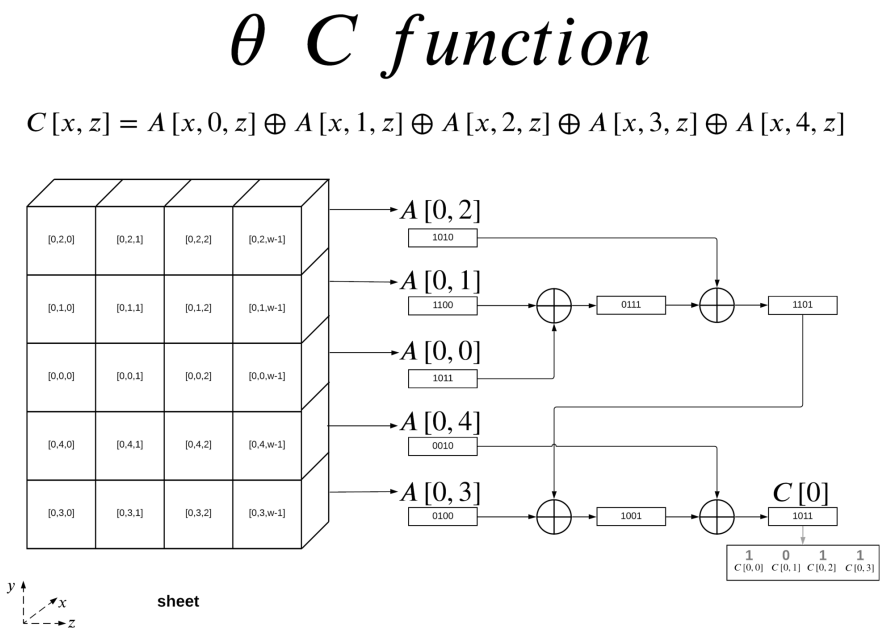
Bước θ (theta) đóng vai trò tạo ra sự khuếch tán dữ liệu, đảm bảo một thay đổi nhỏ ở một bit bất kỳ sẽ nhanh chóng lan tỏa và ảnh hưởng đến nhiều bit khác trong state.

Cơ chế: Mỗi bit trong state được cập nhật bằng cách XOR với parity (tính chẵn lẻ) của column (x-1, z) và parity của column (x+1, z-1)

Bước θ bao gồm hai hàm riêng biệt: hàm C và hàm D

➢ **Hàm C của θ**

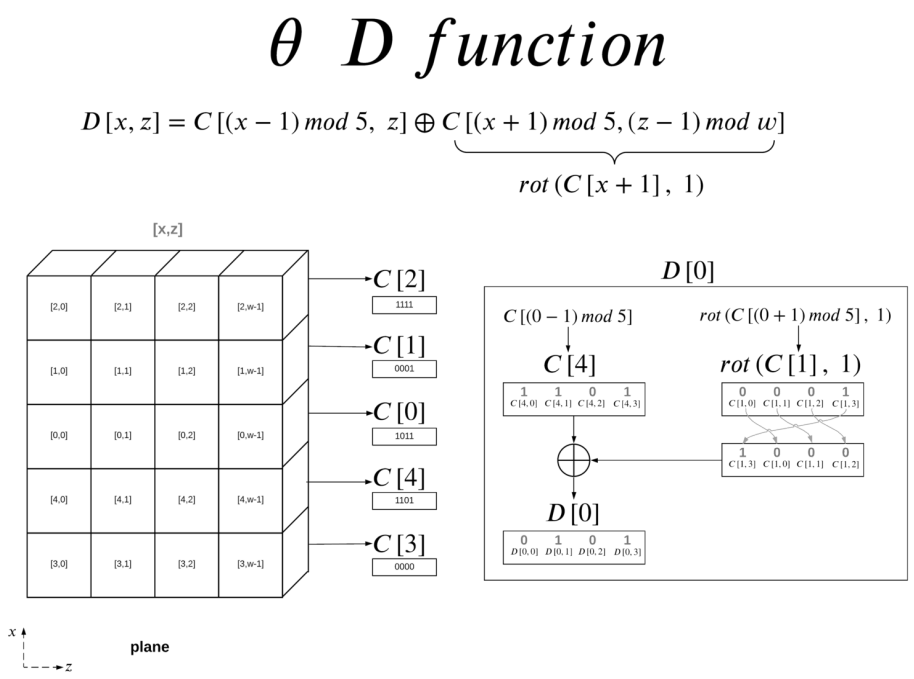
Là hàm tính toán tính chẵn lẻ (parity) cho từng column. Việc XOR tất cả các bit trong một column giúp "nén" dữ liệu từ State 3D thành một plane 2D (kích thước 5 x 64).



Hình 14 - Trực quan hóa hàm C của θ

➢ **Hàm D của θ**

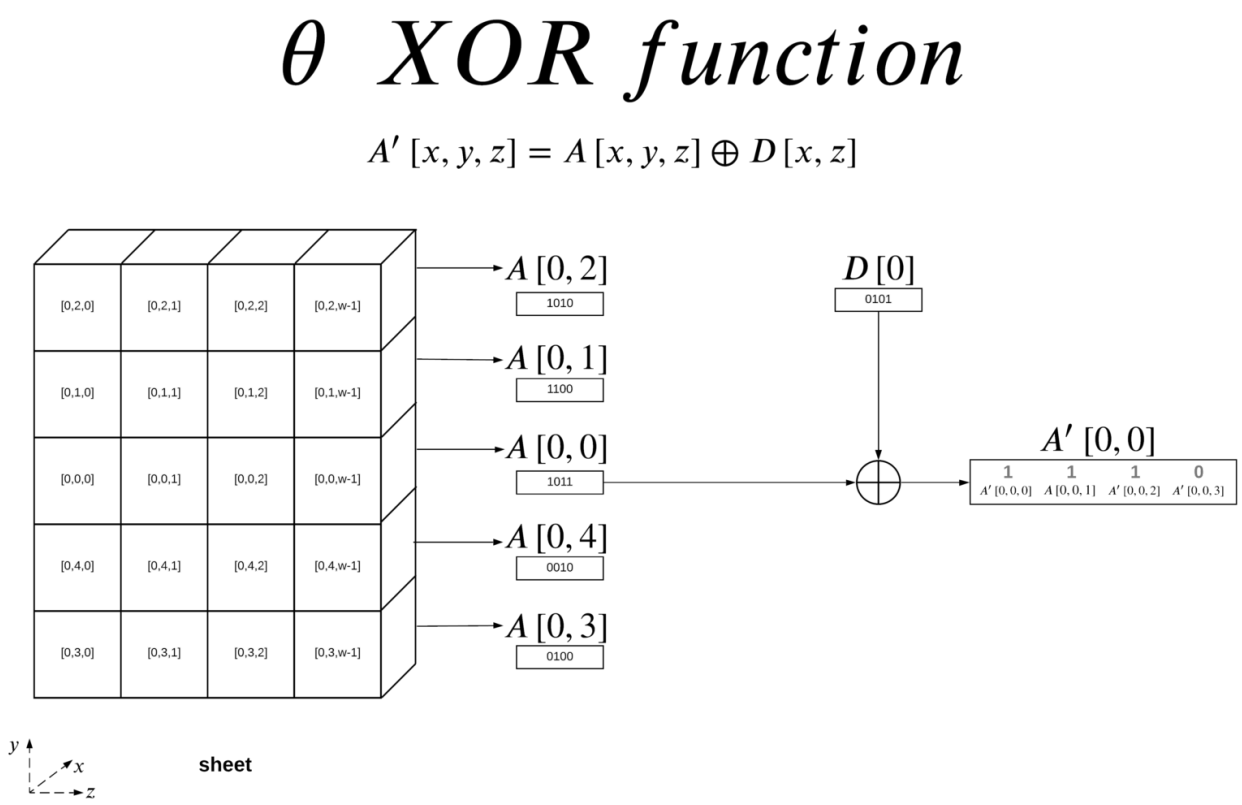
Để tính giá trị điều chỉnh cho một column, hàm D kết hợp parity của column bên trái (x-1) và parity của column bên phải (x+1) đã được xoay 1 bit dọc theo lane (trục z)



Hình 15 - Trực quan hóa hàm D của θ

➢ **Đầu ra của θ**

Đầu ra của hàm θ là kết quả XOR giữa mảng trạng thái ban đầu A và đầu ra của hàm D.



Hình 16 - Trực quan hóa phép XOR đầu ra của θ

Đầu ra của hàm được ký hiệu là A′, có kích thước 5 × 5 × 64 bit, giống với kích thước của state ban đầu. Sau đó A′ được đưa vào bước ρ (rho).

5.3.3.3. Bước ρ (Rho)

Bước ρ (rho) đóng vai trò phân tán dữ liệu theo trục lane. Nếu không có bước này, các bit tại cùng một vị trí z trên các lane khác nhau sẽ không bao giờ thay đổi vị trí tương đối của chúng.

Cơ chế: Mỗi lane tại tọa độ (x, y) được xoay vòng các bit của nó theo trục z với một giá trị dịch chuyển (offset) cố định được xác định trước tùy theo vị trí của lane đó trong state

ρ function

$$A'[0,0] = A[0,0]$$

for $t = 0$ to 23 **do**

$$A'[x,y] = ROT\left(A[x,y],\ \rho_{of}\ mod\ w\right)$$

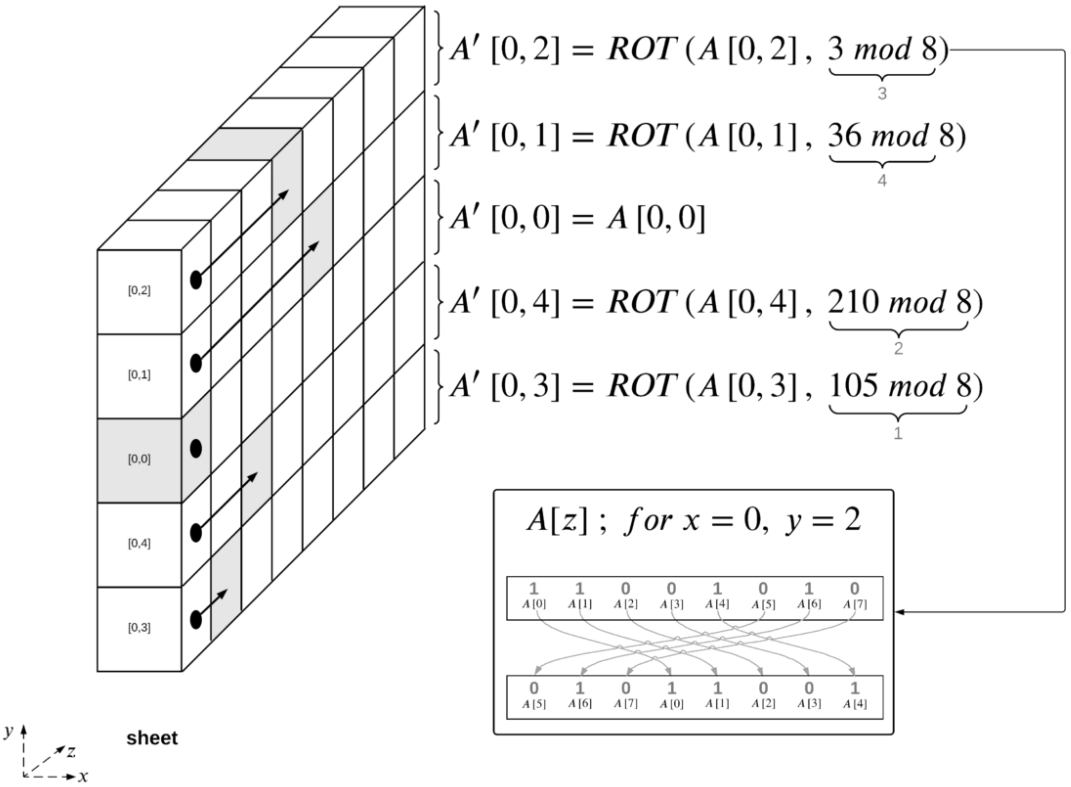
$$\rho_{of} = \frac{(t+1)(t+2)}{2}$$

$$\begin{pmatrix} i \\ j \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}^t \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$x = i\ mod\ 5$$
$$y = j\ mod\ 5$$

Table of ρ_{of}

	$x=3$	$x=4$	$x=0$	$x=1$	$x=2$
$y=2$	153	231	3	10	171
$y=1$	55	276	36	300	6
$y=0$	28	91	0	1	190
$y=4$	120	78	210	66	253
$y=3$	21	136	105	45	15



Hình 17 - Trực quan hóa hàm ρ trên một sheet của state.

Các giá trị xoay không phải ngẫu nhiên mà tuân theo các số tam giác (0, 1, 3, 6, 10, ...). Việc tính toán tọa độ (x, y) cho từng offset dựa trên phép nhân ma trận đặc thù của Keccak.

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A^2 = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} aa + bc & ab + bd \\ ca + dc & cb + dd \end{pmatrix}$$

$$A^3 = A^2 \cdot A = \begin{pmatrix} a^2 + bc & ab + bd \\ ca + dc & cb + d^2 \end{pmatrix} \cdot \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A^n = A^{n-1} \cdot A$$

Hình 18 - Minh họa phép nhân ma trận

$$t = 10$$

1. Find number of rotations

$$\rho_{of}(10) = \frac{(10+1)(10+2)}{2} = 66$$

2. Compute x and y

$$\begin{pmatrix} i \\ j \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}^{10} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 44726 \\ 159294 \end{pmatrix}$$

$$x = 44726 \ mod \ 5 = 1$$
$$y = 159294 \ mod \ 5 = 4$$

3. Validate with table

$$x = 1, \ y = 4$$
$$\rho_{of} = 66$$

Table of ρ_{of}

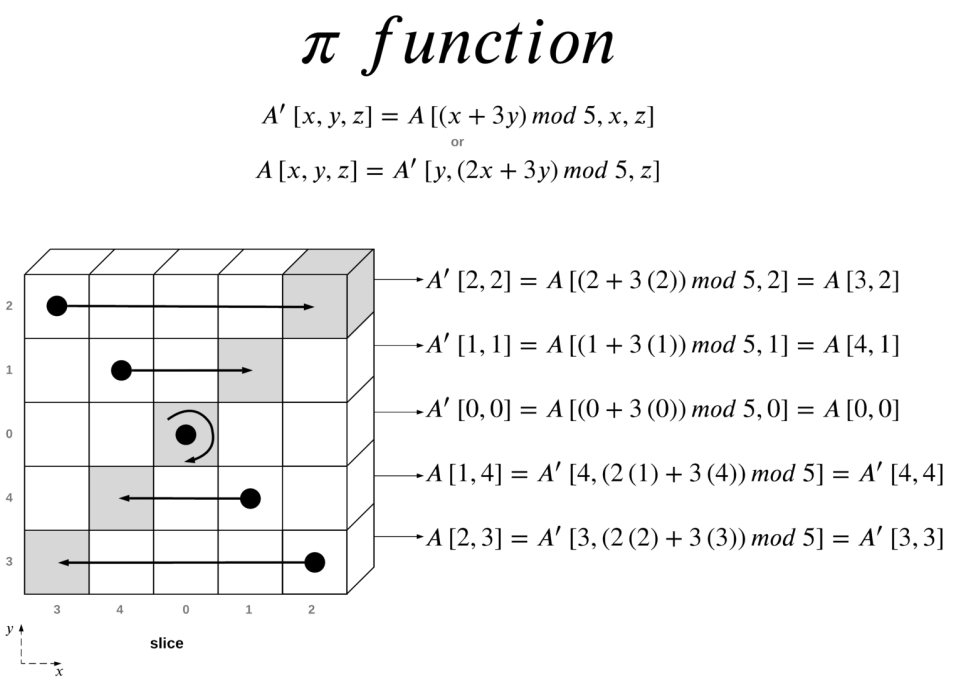
	$x = 3$	$x = 4$	$x = 0$	$x = 1$	$x = 2$
$y = 2$	153	231	3	10	171
$y = 1$	55	276	36	300	6
$y = 0$	28	91	0	1	190
$y = 4$	120	78	210	66	253
$y = 3$	21	136	105	45	15

Hình 19 - Suy diễn bảng offset của bước ρ.

5.3.3.4. Bước π (Pi)

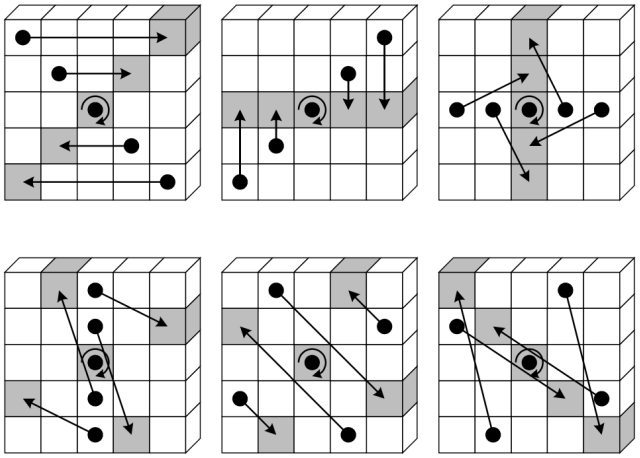
Bước π (pi) đóng vai trò xáo trộn vị trí của các lane trong mỗi slice để phá vỡ các cấu trúc tuyến tính và tăng cường khả năng khuếch tán dữ liệu trên toàn bộ mảng trạng thái.

Cơ chế: Mỗi lane tại vị trí (x, y) được di chuyển đến một vị trí mới (x', y') trong cùng một slice theo công thức hoán vị tọa độ: x' = y và y' = (2x + 3y) mod 5



Hình 20 - Trực quan hóa hàm π trên một slice.

Ý nghĩa: Phép hoán vị này giúp phá vỡ các cấu trúc hàng/cột cố định, ngăn chặn các cuộc tấn công phân tích mật mã dựa trên tính đối xứng của ma trận State



Hình 21 - Sơ đồ hàm π trong tiêu chuẩn FIPS 202

5.3.3.5. Bước χ (Chi)

Bước χ (chi) là bước phi tuyến tính duy nhất trong hàm hoán vị, đóng vai trò tạo ra sự xáo trộn khiến mối quan hệ giữa các bit đầu vào và đầu ra trở nên cực kỳ phức tạp.

Cơ chế: Mỗi bit trong một row được cập nhật bằng cách XOR với (NOT bit_{x+1} AND bit_{x+2}).

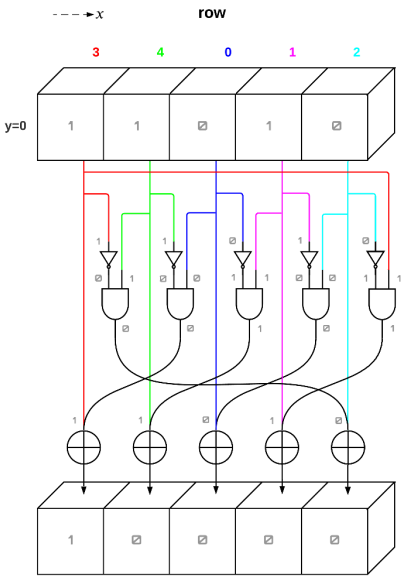
χ đóng vai trò như một hộp thay thế khổng lồ. Vì nó chỉ hoạt động theo hướng các row, nên bước π trước đó là bắt buộc để đảm bảo sự khuếch tán đã được trải đều trước khi thực hiện tính toán phi tuyến này.

$$\chi \ function$$

for $y = 0$ to 4 **do**

for $x = 0$ to 4 **do**

$$A'[x, y] = A[x, y] \oplus ((NOT\ A[(x+1)\ mod\ 5, y])\ AND\ A[(x+2)\ mod\ 5, y])$$



$$A'[3] = A[3] \oplus ((NOT\ A[(3+1) mod\ 5])\ AND\ A[(3+2) mod\ 5])$$

$$A'[3] = \underbrace{\underbrace{A[3]}_{1} \oplus \underbrace{\underbrace{(NOT\ \underbrace{A[4]}_{1})}_{0}\ AND\ \underbrace{A[0]}_{0}}_{0}}_{1}$$

$$A'[3] = 1$$

Hình 22 - Trực quan hóa hàm χ trên một hàng.

5.3.3.6. Bước ι (Iota)

Bước ι (iota) đóng vai trò phá vỡ tính đối xứng của các vòng lặp bằng cách thêm một hằng số vòng (round constant) khác nhau vào mỗi vòng, ngăn chặn các cuộc tấn công dựa trên tính chất lặp lại.

Cơ chế: XOR một hằng số vòng (Round Constant - RC) vào duy nhất lane tại tọa độ (0,0). Toàn bộ 24 lane còn lại không thay đổi.

ι function

$$A'[x,y,z] = A[x,y,z] \oplus RC[i_r]$$

$$RC[i_r][0][0]\left[2^j - 1\right] \; for\ all\ 0 \le j \le \ell$$

$\ell = 6$

j	[z]
0	0
1	1
2	3
3	7
4	15
5	31
6	63

The Round Constant can only set 7 bits

This function only changes these 7 bits in the center lane

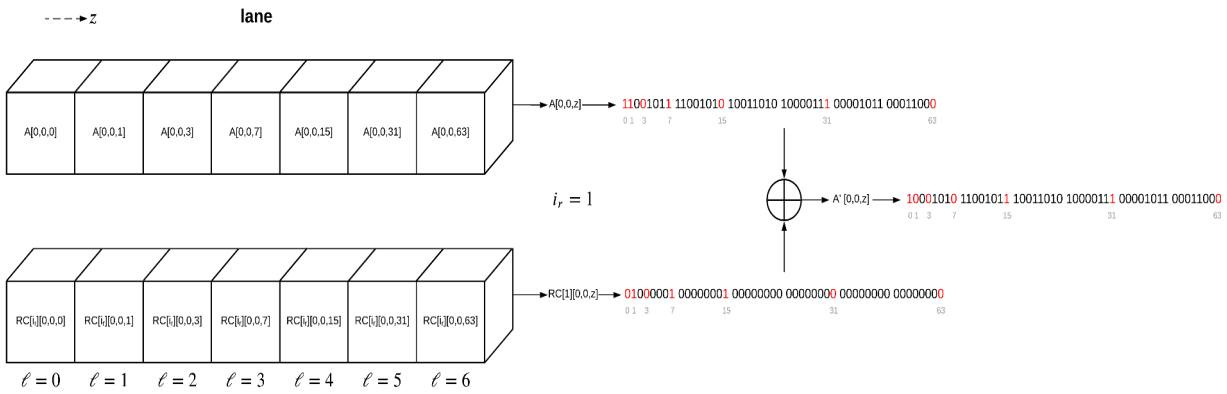


Table 1: The round constants RC[i]

RC[0]	0x0000000000000001	RC[12]	0x000000008000808B
RC[1]	0x0000000000008082	RC[13]	0x800000000000008B
RC[2]	0x800000000000808A	RC[14]	0x8000000000008089
RC[3]	0x8000000080008000	RC[15]	0x8000000000008003
RC[4]	0x000000000000808B	RC[16]	0x8000000000008002
RC[5]	0x0000000080000001	RC[17]	0x8000000000000080
RC[6]	0x8000000080008081	RC[18]	0x000000000000800A
RC[7]	0x8000000000008009	RC[19]	0x800000008000000A
RC[8]	0x000000000000008A	RC[20]	0x8000000080008081
RC[9]	0x0000000000000088	RC[21]	0x8000000000008080
RC[10]	0x0000000080008009	RC[22]	0x0000000080000001
RC[11]	0x000000008000000A	RC[23]	0x8000000080008008

Hình 23: Trực quan hóa hàm ι trên một lane.

Sinh hằng số RC: Các hằng số này được tạo ra bởi một bộ tạo số giả ngẫu nhiên LFSR (Linear Feedback Shift Register). Việc mỗi vòng lặp (r = 0…23) sử dụng một RC khác nhau khiến cho mỗi lần biến đổi State là độc nhất.

[RC] function

$$RC[i_r][0][0]\underbrace{[2^j - 1]}_{\text{bit \#}} = rc[\underbrace{j + 7i_r}_{t}] \quad for\ all\ 0 \le j \le \ell$$

$$\ell = 6$$

j	bit #	t
0	0	$0+7i_r$
1	1	$1+7i_r$
2	3	$2+7i_r$
3	7	$3+7i_r$
4	15	$4+7i_r$
5	31	$5+7i_r$
6	63	$6+7i_r$

$$rc[t] = (x^t\ mod\ x^8 + x^6 + x^5 + x^4 + 1) mod\ x \in GF(2)[x]$$

Linear Feedback Shift Register Polynomial

$$x^8 + x^6 + x^5 + x^4 + 1$$

1. $b_t^1 = b_{t-1}^8, \quad b_t^5 = b_{t-1}^8 \oplus b_{t-1}^4, \quad b_t^6 = b_{t-1}^8 \oplus b_{t-1}^5, \quad b_t^7 = b_{t-1}^8 \oplus b_{t-1}^6$

Tapped Bits are **XORed** with 8th bit then shifted right.

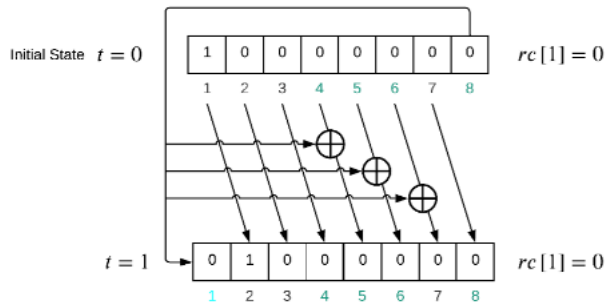
2. $rc(t) = b_t^1$

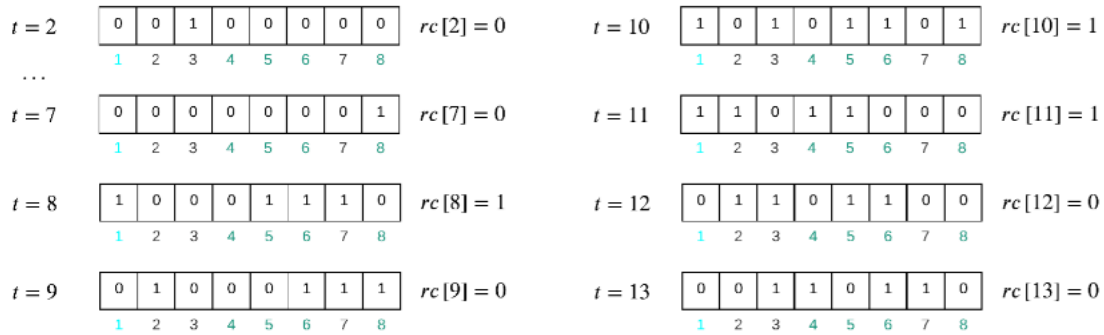
rc output is current state's 1st bit.

Table 1: The round constants RC[i]

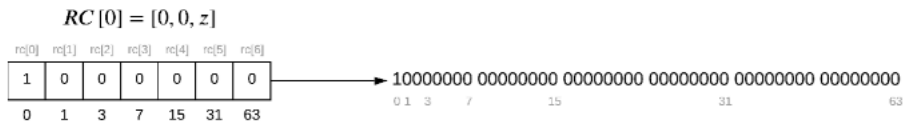
RC[0]	0x0000000000000001	RC[12]	0x000000008000808B
RC[1]	0x0000000000008082	RC[13]	0x800000000000008B
RC[2]	0x800000000000808A	RC[14]	0x8000000000008089
RC[3]	0x8000000080008000	RC[15]	0x8000000000008003
RC[4]	0x000000000000808B	RC[16]	0x8000000000008002
RC[5]	0x0000000080000001	RC[17]	0x8000000000000080
RC[6]	0x8000000080008081	RC[18]	0x000000000000800A
RC[7]	0x8000000000008009	RC[19]	0x800000008000000A
RC[8]	0x000000000000008A	RC[20]	0x8000000080008081
RC[9]	0x0000000000000088	RC[21]	0x8000000000008080
RC[10]	0x0000000080008009	RC[22]	0x0000000080000001
RC[11]	0x000000008000000A	RC[23]	0x8000000080008008

Reversed Galois LFSR

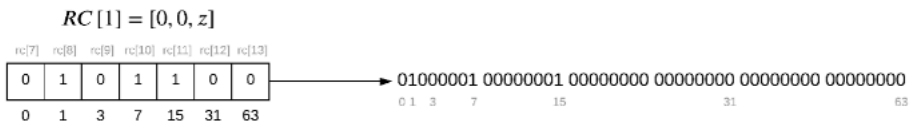




i_r	t	bit #	rc[t]
0	0	0	1
0	1	1	0
0	2	3	0
0	3	7	0
0	4	15	0
0	5	31	0
0	6	63	0



i_r	t	bit #	rc[t]
1	7	0	0
1	8	1	1
1	9	3	0
1	10	7	1
1	11	15	1
1	12	31	0
1	13	63	0



Hình 24 - Minh họa quá trình sinh Round Constant.

5.3.4. Giai đoạn “Vắt"

Sau giai đoạn Hấp thụ, State 1600 bit đã chứa toàn bộ thông tin của dữ liệu đầu vào. Giai đoạn “Vắt” (Squeezing Phase) thực hiện trích xuất các bit từ State để tạo ra bản băm cuối cùng

➢ **SHA-3 với độ dài cố định (SHA3-224, SHA3-256, SHA3-384, SHA3-512)**

Do độ dài đầu ra (d) của các phiên bản này luôn nhỏ hơn tốc độ hấp thụ (r), quá trình vắt chỉ diễn ra một lần duy nhất:

Quy trình thực hiện:

- Lấy d bit đầu tiên của state sau khi kết thúc giai đoạn absorbing.
- Chuỗi d bit này là chuỗi bit đầu ra của thuật toán. Quá trình kết thúc

➢ **SHA-3 với độ dài đầu ra tùy chỉnh (SHAKE128 và SHAKE256)**

SHAKE cho phép tạo ra đầu ra với độ dài bất kỳ theo yêu cầu người dùng. Do đó, thuật toán có thể lặp lại quá trình squeezing nhiều lần để tạo thêm bit.

Quy trình thực hiện:

- Khởi tạo một mảng bit rỗng để chứa kết quả đầu ra.
- Lặp lại các bước sau cho đến khi thu được đủ số bit yêu cầu:
    - Bước 1: Trích xuất r bit đầu tiên từ state và ghép vào cuối mảng kết quả.
    - Bước 2: Kiểm tra độ dài kết quả
- Nếu đã thu được đủ số bit yêu cầu → dừng quá trình và trả về kết quả
- Nếu chưa đủ → Áp dụng hàm hoán vị f (24 vòng Keccak) lên State để tạo ra một trạng thái mới hoàn toàn, sau đó quay lại Bước 1 để trích xuất tiếp r bit tiếp theo.
    - Bước 3: Cắt kết quả: Sau khi dừng, nếu tổng số bit vượt quá yêu cầu, thuật toán thực hiện cắt bỏ các bit thừa ở cuối để trả về đúng d bit đầu tiên.

5.3.5. Xử lý và biểu diễn kết quả đầu ra

Quy trình thực hiện:

1. Nhóm chuỗi bit đầu ra thành các byte, mỗi byte gồm 8 bit
2. Đảo thứ tự bit trong mỗi byte (chuyển sang dạng MSB-first)
3. Chuyển các byte sang dạng hexadecimal (hex)

Chuỗi hexadecimal thu được chính là giá trị hash cuối cùng của thuật toán.

Ví dụ mã băm SHA3-256 cho chuỗi “Hello world!”:

d6ea8f9a1f22e1298e5a9506bd066f23cc56001f5d36582344a628649df53ae8

5.4. Demo

https://nvbangg.github.io/demo-sha/

5.5 Kết chương

Chương 5 đã cung cấp một cái nhìn toàn diện về SHA-3 được chuẩn hóa theo FIPS PUB 202. Qua việc phân tích cấu trúc và cơ chế vận hành, chúng ta có thể rút ra các đặc điểm cốt lõi sau:

- Sự đột phá về kiến trúc: Khác với các thế hệ SHA trước đó dựa trên cấu trúc Merkle-Damgård, SHA-3 sử dụng cấu trúc Bọt biển (Sponge Construction). Cơ chế này chia làm hai giai đoạn: Hấp thụ (Absorbing) để nạp dữ liệu vào trạng thái nội bộ và Vắt (Squeezing) để trích xuất giá trị băm với độ dài mong muốn.
- Trạng thái nội bộ mạnh mẽ: Giải thuật vận hành trên một ma trận 3 chiều 5 x 5 x 64 bit gọi là State, tổng cộng 1600 bit đối với SHA-3. Quá trình biến đổi được thực hiện qua 24 vòng lặp với 5 bước toán học theta, rho, pi, chi, iota giúp đảm bảo tính khuếch tán và phi tuyến tính cực cao.
- Tính linh hoạt vượt trội: SHA-3 không chỉ cung cấp các phiên bản có độ dài đầu ra cố định (SHA3-224 đến SHA3-512) mà còn giới thiệu các hàm đầu ra có thể mở rộng (XOFs) như SHAKE128 và SHAKE256. Điều này cho phép tùy chỉnh độ dài băm linh hoạt cho nhiều ứng dụng khác nhau.
- Ưu thế bảo mật: Nhờ thiết kế khác biệt hoàn toàn, SHA-3 miễn nhiễm với các cuộc tấn công mở rộng độ dài (Length Extension Attack) – một điểm yếu cố hữu của SHA-2. Đây là một "phương án dự phòng chiến lược" quan trọng, đảm bảo an toàn ngay cả khi các lỗ hổng toán học của cấu trúc cũ bị khai thác.

Tóm lại, SHA-3 không chỉ là một sự nâng cấp về độ dài bit, mà là một bước tiến về tư duy thiết kế mật mã học. Việc nắm vững SHA-3 là cơ sở để tiếp cận các công nghệ hiện đại như Ethereum hay các hệ thống bảo mật hậu lượng tử.

CHƯƠNG 6: CÁC ĐIỂM YẾU, CÁC DẠNG TẤN CÔNG VÀO SHA

6.1. Điểm yếu trong cơ chế mở rộng thông điệp (SHA-0)

Đây là lỗ hổng nguyên bản khiến SHA-0 bị thay thế chỉ sau 2 năm công bố.

- Đối tượng: Duy nhất thuật toán SHA-0.
- Điểm yếu: Quá trình mở rộng thông điệp hoàn toàn tuyến tính. SHA-0 chỉ sử dụng phép XOR để kết hợp các từ (words) mà không có phép xoay bit (Rotation) như trong SHA-1.
- Hệ quả: Độ khuếch tán (Diffusion) kém: Sự thay đổi ở đầu vào không lan tỏa đủ mạnh ra toàn bộ mã băm, khiến mối quan hệ giữa các từ trở nên dễ dự đoán.
    - Tạo đường vi sai (Differential Path): Kẻ tấn công có thể dễ dàng xây dựng các quỹ đạo sai khác để tìm ra va chạm với chi phí cực thấp.
- Kết luận: SHA-0 không còn đảm bảo bất kỳ tính chất bảo mật nào và đã bị khai tử hoàn toàn khỏi các ứng dụng thực tế.

6.2. Các dạng tấn công vào SHA-0

Phân tích vi sai (Differential Cryptanalysis):

- Khai thác sự thiếu hụt khuếch tán để theo dõi sai khác giữa các thông điệp.
- Hiệu quả: Giảm độ phức tạp tìm va chạm từ 2^80 xuống còn 2^39.

Vector nhiễu & Va chạm cục bộ (Perturbation Vector & Local Collision):

- Tận dụng tính tuyến tính để chèn sai lệch vào các bước lặp và triệt tiêu chúng ở vòng sau.
- Hiệu quả: Tăng đáng kể xác suất tạo ra va chạm mã băm.

Sửa đổi thông điệp (Message Modification):

- Điều chỉnh trực tiếp các bit đầu vào để ép kết quả đầu ra khớp với mục tiêu vi sai.
- Hiệu quả: Nâng xác suất tấn công thành công lên mức gần như tuyệt đối (xấp xỉ 1).

6.3. Tấn công Va chạm (Collision Attack) - "Tử huyệt" của SHA-1

Đây là dạng tấn công tìm hai đầu vào khác nhau nhưng cùng một kết quả mã băm đầu ra.

- Đối tượng: Chủ yếu là SHA-1.
- Điểm yếu: Cấu trúc toán học của SHA-1 cho phép kẻ tấn công tìm ra va chạm nhanh hơn nhiều so với lý thuyết (nhanh hơn 2^80 phép thử).
- Thực tế: Năm 2017, Google công bố cuộc tấn công SHAttered, tạo ra 2 file PDF có nội dung khác nhau nhưng cùng mã băm SHA-1.
- Hệ quả: SHA-1 bị loại bỏ khỏi các chứng chỉ SSL/TLS và chữ ký số vì kẻ xấu có thể giả mạo tài liệu mà chữ ký vẫn khớp.

6.4. Tấn công Mở rộng độ dài (Length Extension Attack) - Điểm yếu của SHA-2

Đây là lỗi thiết kế đặc trưng của cấu trúc Merkle–Damgård.

- Đối tượng: SHA-2 (SHA-256, SHA-512).
- Điểm yếu: Nếu ta biết Hash(M), ta có thể tính được Hash(M || Extra) mà không cần biết nội dung của M. Điều này cực kỳ nguy hiểm nếu hệ thống dùng công thức Hash(Secret || message) để xác thực. Kẻ tấn công có thể: Hash (message || padding || extra) mà không cần biết secret.
- Tại sao SHA-3 an toàn? SHA-3 sử dụng cấu trúc Sponge (miếng bọt biển). Nó có một cơ chế "xả" (squeeze) dữ liệu giúp che giấu trạng thái nội tại, khiến kẻ tấn công không thể nối thêm dữ liệu vào cuối chuỗi băm.

6.5. Tấn công Vét cạn và Bảng cầu vồng (Brute Force & Rainbow Tables)

Tấn công vào tính chất "kháng tiền ảnh" (Pre-image resistance).

- Đối tượng: Tất cả các loại SHA (khi dùng để băm mật khẩu).
- Điểm yếu: Các hàm SHA được thiết kế để chạy rất nhanh. Điều này vô tình giúp kẻ tấn công có thể thử hàng tỷ mật khẩu mỗi giây.
- Công cụ: Sử dụng GPU hoặc máy đào ASIC (vốn dùng để đào Bitcoin) để bẻ khóa mật khẩu cực nhanh.
- Phòng chống: Bắt buộc phải dùng Salt (muối) và các hàm "chậm" như PBKDF2 hoặc Argon2 thay vì dùng SHA trực tiếp cho mật khẩu.

6.6. Tấn công Kênh bên (Side-Channel Attack)

Tấn công vào quá trình thực thi vật lý thay vì toán học.

- Đối tượng: Các thiết bị phần cứng (Smart Card, Ví lạnh Crypto) chạy SHA.
- Điểm yếu: Khi chip xử lý thuật toán SHA, nó sẽ rò rỉ thông tin qua:
    - Điện năng tiêu thụ: Biến thiên dòng điện tiết lộ các bước tính toán.
    - Thời gian: Thời gian xử lý khác nhau giữa các khối dữ liệu.
    - Bức xạ điện từ.
- Kết quả: Kẻ tấn công có thể khôi phục lại các khóa bí mật được lưu bên trong chip.

Bảng 2. Bảng tổng hợp khả năng chống chịu tấn công

Dạng tấn công	SHA-1	SHA-2 (256/512)	SHA-3
Collision (Va chạm)	Đã bị phá	An toàn (Lý thuyết 2^128)	Rất an toàn
Length Extension	Bị ảnh hưởng	Bị ảnh hưởng	Miễn nhiễm
Pre-image (Vét cạn)	Yếu (do đầu ra ngắn)	Rất an toàn	Rất an toàn
Sức mạnh tính toán	Thấp	Trung bình (Nhanh trên CPU)	Cao (Tối ưu trên phần cứng)

6.7. Kết chương

1. SHA-1: Tuyệt đối không dùng cho các mục đích bảo mật, chỉ dùng để kiểm tra lỗi dữ liệu (checksum) không quan trọng.
2. SHA-2: Vẫn là tiêu chuẩn công nghiệp nhưng phải dùng kèm với HMAC hoặc các kỹ thuật bao bọc khác để chống tấn công mở rộng độ dài.
3. SHA-3: Là tương lai của mật mã học. Nếu bạn đang thiết kế một hệ thống mới từ đầu, SHA-3 là lựa chọn sáng suốt nhất để "đi trước một bước" so với các dạng tấn công hiện đại.

CHƯƠNG 7: CÀI ĐẶT VÀ THỬ NGHIỆM MÃ HÓA VÀ THÁM MÃ CÁC GIẢI THUẬT SHA

Trong phần này, ta sẽ thực hiện cài đặt và thử nghiệm thực tế các giải thuật băm SHA-0, SHA-1, SHA-2 và SHA-3 sử dụng ngôn ngữ lập trình Python với thư viện hashlib (thư viện tiêu chuẩn hỗ trợ các hàm băm mật mã). Vì SHA-0 không được hỗ trợ trực tiếp trong hashlib (do đã bị loại bỏ khỏi các tiêu chuẩn), ta sẽ mô phỏng hoặc sử dụng các triển khai tương đương nếu cần, nhưng tập trung vào các phiên bản còn lại. Thử nghiệm bao gồm:

- Mã hóa: Tính toán giá trị băm cho các dữ liệu đầu vào mẫu, so sánh độ dài và tính chất của chúng.
- Thám mã: Thực hiện các tấn công mẫu như brute-force (tấn công vét cạn), rainbow table (bảng cầu vồng đơn giản), và collision attack (tấn công va chạm). Do tính phức tạp tính toán, chúng tôi sẽ demo trên quy mô nhỏ (mật khẩu ngắn) và phân tích tính khả thi thực tế.

Thử nghiệm được thực hiện trên môi trường Python 3.12.3, không yêu cầu cài đặt thêm gói nào. Kết quả sẽ được trình bày dưới dạng mã nguồn, kết quả thực tế, phân tích.

7.1. Cài Đặt Môi Trường Thử Nghiệm

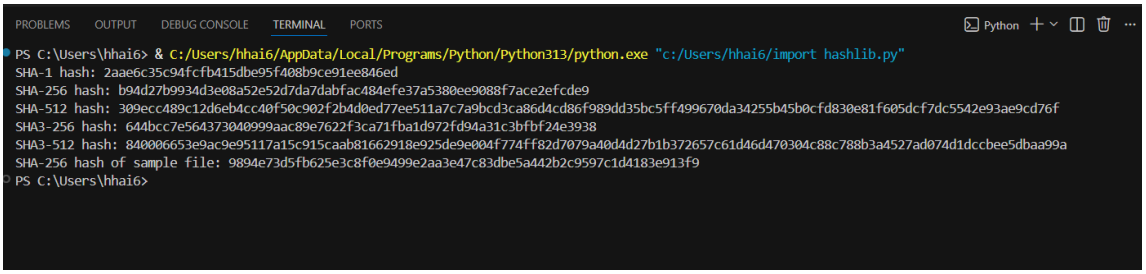
- Công cụ: Python với hashlib cho mã hóa. Các script được viết đơn giản để dễ tái hiện.
- Dữ liệu đầu vào mẫu:
    - Chuỗi văn bản: "hello world" (dữ liệu đơn giản).
    - Mật khẩu mẫu cho thám mã: Các chuỗi ngắn (1-4 ký tự) để demo brute-force khả thi.
    - Dữ liệu mẫu: Tạo một file văn bản nhỏ để hash toàn bộ file.
- Tiêu chí đánh giá: Thời gian thực thi, độ dài hash, tính kháng tấn công (dựa trên kết quả cracking).

7.2. Thử Nghiệm Mã Hóa (Hashing)

Ta tính hash cho dữ liệu đầu vào bằng các phiên bản SHA. SHA-0 không có sẵn, nhưng có thể mô phỏng bằng cách sử dụng SHA-1 với các điều chỉnh lịch sử (SHA-0 giống SHA-1 trừ một phép xoay bit bị thiếu). Tuy nhiên, để chính xác, ta sử dụng SHA-1 làm đại diện gần nhất và lưu ý sự khác biệt.

7.2.1. Mã Nguồn và Thực Thi Hashing

Dưới đây là script Python để tính hash. Chúng tôi đã chạy script này trong môi trường Visual Studio Code để lấy kết quả thực tế.



Hình 25 - Kết quả của thử nghiệm mã hóa

7.2.2. Phân Tích Kết Quả Mã Hóa

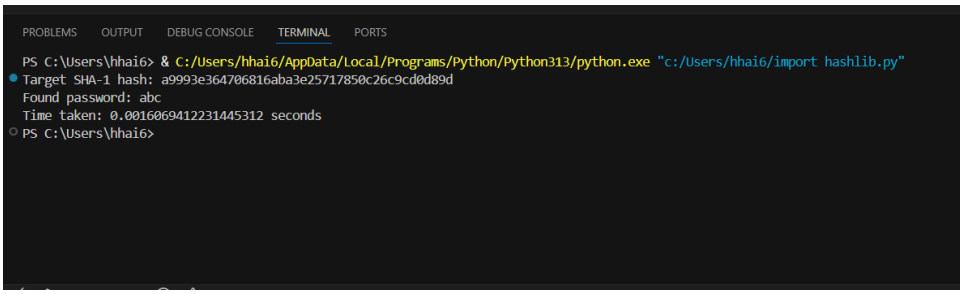
- Độ dài hash: SHA-1: 160 bit (40 ký tự hex); SHA-256: 256 bit (64 hex); SHA-512: 512 bit (128 hex); SHA-3 tương tự.
- Tính avalanche effect: Thay đổi 1 bit input dẫn đến thay đổi lớn ở output. Ví dụ, hash "hello world!" (thêm dấu chấm than):
  - SHA-256: 7509e5bda0c762d2bac7f90d758b5b2263fa01ccbc542ab5e3df163be08e6ca9 (khác hoàn toàn so với bản gốc).
- Thời gian thực thi: Trên môi trường tiêu chuẩn, hashing một chuỗi ngắn mất <1ms. Đối với file lớn (ví dụ 1MB), SHA-3 chậm hơn SHA-2 khoảng 20-30% do cấu trúc sponge.

7.3. Thử Nghiệm Thám Mã (Cracking)

Hash không thể "giải mã" trực tiếp (one-way), nên cracking tập trung vào tìm input khớp hash qua tấn công. Chúng tôi demo brute-force và rainbow table đơn giản cho SHA-1 (dễ tấn công nhất), và collision cho SHA-1.

7.3.1. Brute-Force Attack

- **Mô tả**: Thử tất cả input có thể (ví dụ alphabet a-z, 1-4 ký tự). Không khả thi cho password dài, nhưng demo cho thấy chi phí.



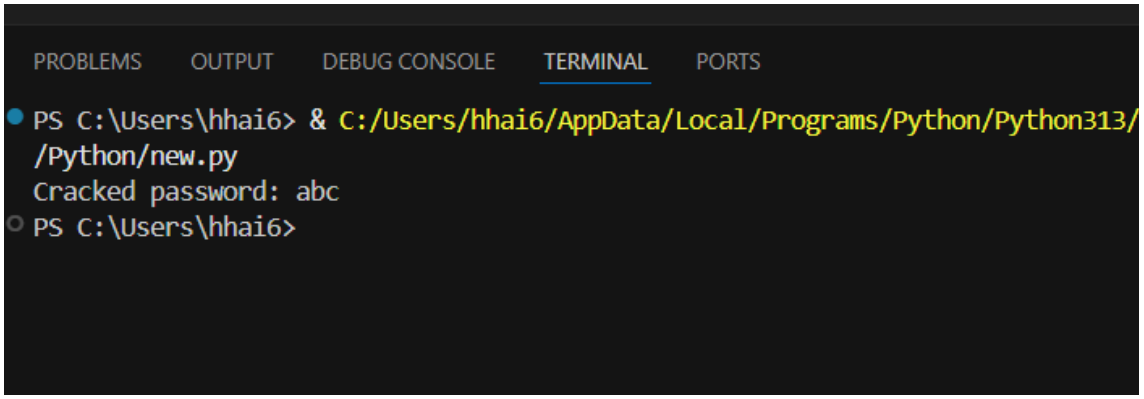
Hình 26 - kết quả của thử nghiệm thám mã brute-force

Kết quả thực thi:

- Target SHA-1 hash: a9993e364706816aba3e25717850c26c9cd0d89d
- Found password: abc
- Time taken: 0.0016069412231445312 seconds
- Phân tích: Với password 8 ký tự (a-z, số, ký tự đặc biệt: 95^8 ≈ 6.6e15), cần ~200 năm trên GPU mạnh (10^9 hash/s). SHA-2/SHA-3 kháng tốt hơn do độ dài lớn.

7.3.2. Rainbow Table Attack

- Mô tả: Lưu trữ bảng pre-computed hash để tra cứu nhanh. Demo đơn giản với dict Python.



Hình 27 - kết quả của thử nghiệm thám mã rainbow table

Kết quả thực thi:

- Cracked password: abc
- Phân tích: Rainbow table hiệu quả cho password yếu, nhưng cần bộ nhớ lớn (terabytes cho bảng đầy đủ). Chống bằng salt (thêm chuỗi ngẫu nhiên: hash(password + salt)).

7.3.3. Collision Attack

- Mô tả: Tìm hai input khác nhau có cùng hash. Không thể tạo mới (vì cần tài nguyên lớn) nên ta sử dụng ví dụ thực tế SHAttered (dự án hợp tác giữa Google và CWI năm 2017): Hai file PDF có cùng SHA-1 hash.
- Demo kiểm tra: Sử dụng lệnh terminal (hoặc Python) để xác nhận. Giả sử tải file từ shattered.io:
    - File 1: shattered-1.pdf (hash: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a)
    - File 2: shattered-2.pdf (cùng hash, nhưng nội dung khác).

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Python
PS C:\Users\hhai6> & C:/Users/hhai6/AppData/Local/Programs/Python/Python313/python.exe c:/Users/hhai6/One
/Python/new.py
Hash1: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a
Hash2: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a
Collision: True
PS C:\Users\hhai6>
```

Hình 28 - kết quả của thử nghiệm thám mã collision

Kết quả thực thi:

Collision: True.

- Phân tích: Tấn công này mất 110 GPU-years, chứng minh SHA-1 không an toàn cho chứng chỉ. SHA-2/SHA-3 chưa có collision thực tế.

7.4. Kết chương

Từ kết quả thử nghiệm, SHA-1 dễ bị tấn công brute-force và collision ngay cả trên quy mô nhỏ, trong khi SHA-2 và đặc biệt SHA-3 cho thấy khả năng kháng cự tốt hơn đáng kể. Thời gian cracking tăng theo cấp số nhân khi độ dài input (mật khẩu) tăng lên, chứng tỏ việc sử dụng mật khẩu dài và thuật toán băm mạnh là yếu tố then chốt.

Giải pháp phòng chống:

1. Sử dụng salt và vòng lặp: Áp dụng PBKDF2 (Password-Based Key Derivation Function 2) với SHA-256, khuyến nghị sử dụng ít nhất 100.000–600.000 vòng lặp (tùy theo khả năng phần cứng server). Mục tiêu là làm cho mỗi lần thử mật khẩu mất khoảng 100–500 ms trên CPU thông thường, khiến tấn công brute-force hoặc dictionary trở nên cực kỳ tốn thời gian và tài nguyên.
2. Chuyển sang Argon2 hoặc bcrypt: Ưu tiên Argon2id (memory-hard, kháng GPU/ASIC tốt nhất hiện nay) với thông số ví dụ: memory=64 MiB, time=1–3, parallelism=4. Nếu hệ thống chưa sẵn sàng Argon2, bcrypt với cost factor 13–17 vẫn là lựa chọn an toàn và được hỗ trợ rộng rãi. Đây là các hàm được OWASP và NIST khuyến nghị hàng đầu cho lưu trữ mật khẩu năm 2026.
3. Giám sát và cập nhật: Định kỳ (hàng quý) sử dụng Hashcat hoặc John the Ripper để kiểm tra nội bộ sức mạnh của các hash hiện có. Đồng thời triển khai di cư dần dần: thêm cột hash mới (Argon2/PBKDF2), re-hash khi người dùng đăng nhập, và sau 3–6 tháng xóa hoàn toàn các hash SHA-1 cũ để loại bỏ hoàn toàn lỗ hổng.
4. Quantum-resistant: Chuẩn bị chuyển dần sang SHA-3 (hoặc SHAKE256) trong các hàm băm và KDF để tăng khả năng kháng quantum (Grover's algorithm). Hiện tại

Argon2 vẫn an toàn đến thập kỷ tới, nhưng việc hỗ trợ SHA-3 trong thư viện (libsodium, PyCryptodome…) sẽ giúp hệ thống sẵn sàng cho tương lai.

5. Best practices: Yêu cầu mật khẩu tối thiểu 12–16 ký tự (ưu tiên passphrase dễ nhớ), bắt buộc kích hoạt đa yếu tố xác thực (2FA/MFA) với TOTP hoặc Passkey (tránh SMS), và sử dụng HMAC-SHA256 (hoặc SHA-3) cho việc xác thực thông điệp, API signature, JWT… nhằm đảm bảo tính toàn vẹn và nguồn gốc dữ liệu.

Thử nghiệm này chứng minh tính cần thiết của việc nâng cấp giải thuật băm để đảm bảo an ninh thông tin.

KẾT LUẬN

1. Các kết quả đạt được

Sau quá trình nghiên cứu nghiêm túc và triển khai thực nghiệm, nhóm em đã thực hiện đề tài "Giải thuật băm SHA 0, 1, 2, 3" hoàn thành đầy đủ các mục tiêu đề ra, đạt được các kết quả cụ thể như sau:

- Về mặt nghiên cứu lý thuyết:
    - Hệ thống hóa toàn diện lộ trình phát triển của họ thuật toán SHA từ phiên bản sơ khai (SHA-0) đến các tiêu chuẩn hiện đại (SHA-1,SHA-2, SHA-3).
    - Phân tích sâu sắc cấu trúc nội tại của từng giải thuật: từ cấu trúc Merkle-Damgård truyền thống đến kiến trúc Sponge (Bọt biển) đột phá của SHA-3. Nhóm đã làm rõ được các hàm logic, cơ chế Message Expansion và Compression Function đặc trưng cho từng loại.
    - Làm rõ các lỗ hổng bảo mật mang tính lịch sử như tính tuyến tính trong SHA-0 và các kỹ thuật tấn công va chạm hiện đại đối với SHA-1, từ đó khẳng định tính tất yếu của việc chuyển dịch sang SHA-2 và SHA-3.
- Về mặt thực nghiệm và Demo:
    - Xây dựng thành công môi trường thử nghiệm: Triển khai cài đặt mã nguồn mô phỏng cơ chế băm của các giải thuật, đảm bảo kết quả đầu ra khớp hoàn toàn với các thư viện chuẩn quốc tế.
    - Phân tích định lượng tính chất mật mã: Thông qua việc tìm hiểu biểu đồ 3D và dữ liệu thực nghiệm, nhóm đã chứng minh được hiệu ứng Avalanche và tiêu chí SAC (Strict Avalanche Criterion). Kết quả cho thấy sự thay đổi dù chỉ 1 bit đầu vào cũng dẫn đến sự biến đổi ngẫu nhiên ~50% các bit đầu ra.
    - Thực nghiệm thám mã: Đã thực hiện demo Collision Attack và tấn công Brute-force trên các phiên bản rút gọn. Đặc biệt, thông qua Birthday Paradox, nhóm đã minh chứng được mối tương quan giữa độ dài bit băm và chi phí tính toán, qua đó khẳng định độ an toàn vượt trội của SHA-256 trong điều kiện hiện nay.

2. Hướng phát triển của đề tài

Dựa trên nền tảng đã đạt được, đề tài có thể được mở rộng và phát triển theo các hướng sau:

- Tối ưu hóa hiệu năng phần cứng: Nghiên cứu triển khai các giải thuật SHA trên thiết bị nhúng (IoT) hoặc FPGA để đánh giá tốc độ xử lý và mức độ tiêu thụ năng lượng.

- Mật mã học hậu kỳ lượng tử: Tìm hiểu khả năng kháng lại các thuật toán lượng tử của SHA-256 và SHA-3 so với các tiêu chuẩn mới đang được NIST dự thảo.
- Ứng dụng thực tế chuyên sâu: Tích hợp các giải thuật này vào hệ thống chữ ký số (Digital Signature) hoặc các cơ chế đồng thuận trong công nghệ Blockchain để giải quyết bài toán toàn vẹn dữ liệu trong giao dịch thực tế.

Bảng 3. So Sánh Chi Tiết Các Thuật Toán Họ SHA

Tiêu chí so sánh	**SHA-0**	**SHA-1**	**SHA-2 (SHA-256/512)**	**SHA-3 (Keccak)**
Năm công bố	1993	1995	2002	2015
Tác giả/Tổ chức	NSA thiết kế, NIST công bố	NSA thiết kế, NIST công bố	NSA thiết kế, NIST công bố	Thuật toán Keccak (thắng giải cuộc thi của NIST)
Cấu trúc nền tảng	Merkle–Damgård	Merkle–Damgård	Merkle–Damgård	Sponge Construction (Cấu trúc bọt biển)
Độ dài giá trị băm (Output)	160	160	224, 256, 384, 512 bit	224, 256, 384, 512 bit (và SHAKE tùy chỉnh)
Kích thước khối (Block size)	512	512	512 bit (SHA-256) hoặc 1024 bit (SHA-512)	Thay đổi tùy theo phiên bản (dựa trên tỷ lệ r)
Số vòng lặp (Steps/Rounds)	80 bước (4 vòng, mỗi vòng 20 bước)	80 bước (4 vòng, mỗi vòng 20 bước)	64 vòng (SHA-256) hoặc 80 vòng (SHA-512)	24 vòng (hàm hoán vị f)

Cơ chế mở rộng thông điệp	Tuyến tính thuần túy (Thiếu phép xoay bit)	Cải tiến bằng cách thêm phép xoay bit (Rotate)	Phi tuyến tính, phức tạp hơn để chống tấn công vi sai	Sử dụng các bước phi tuyến tính
Tình trạng bảo mật	Đã bị phá vỡ hoàn toàn (dễ bị tấn công vi sai)	Không còn an toàn (đã bị tấn công va chạm thực nghiệm năm 2017)	An toàn cao, hiện là tiêu chuẩn vàng cho Blockchain/Bitcoin	Rất an toàn, kháng được các dạng tấn công hiện đại và hậu lượng tử
Khả năng kháng tấn công mở rộng độ dài (Length Extension)	Bị ảnh hưởng	Bị ảnh hưởng	Bị ảnh hưởng (cần dùng kèm HMAC)	Miễn nhiễm hoàn toàn nhờ cấu trúc bọt biển
Ứng dụng thực tế	Đã bị loại bỏ, không còn sử dụng	Chỉ dùng kiểm tra checksum không quan trọng	Phổ biến nhất: Chữ ký số, HTTPS, Bitcoin, ví điện tử	Hệ thống mới cần bảo mật tương lai, đi trước các dạng tấn công hiện đại

TÀI LIỆU THAM KHẢO

[1] FIPS PUB 202

[2] Blog giải thích chi tiết SHA-3

[3] Trang github demo trực quan SHA-3

[4] Collisions on SHA-0 in One Hour , https://link.springer.com/content/pdf/10.1007/978-3-540-71039-4_2.pdf

[5] Bài giảng an toàn và bảo mật hệ thống thông tin - Thầy Hoàng Xuân Dậu.

[6] Y. Motara and B. Irwin, "SHA-1 and the Strict Avalanche Criterion," arXiv preprint arXiv:1609.00616, Sep. 2016.

[7] Chương 4 - FIPS PUB 180-4: Secure Hash Standard (SHS) - National Institute of Standards and Technology (NIST) - Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ, trực thuộc Bộ Thương mại Hoa Kỳ (SHA-2): https://vnshort.com/w4UT